# Algebra and Discrete Mathematics
# ADM_A_B

doc. Xiaolu Hou, PhD.

FIIT, STU
xiaolu.hou @ stuba.sk

# Course Outline

- Vectors and matrices
- System of linear equations
- Matrix inverse and determinants
- Vector spaces, bases and matrix transformations
- Quiz one
- Matrix transformations, fundamental spaces and decompositions
- Eulerian tours and Hamiltonian cycles
- Shortest Paths, Project Scheduling, Trees, and Networks
- Quiz two
- Matching
- Make-up Quizzes

# Recommended reading

- Saoub, K. R. (2017). A tour through graph theory. Chapman and Hall/CRC.
  - Sections 3.1, 3.2, 4.1, 4.2
  - Accessible online (free copy)
  - Alternative download link

# Lecture outline

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Shortest path problem

- A path is a sequence of vertices in which there is an edge between consecutive vertices and no vertex is repeated
- Weight: distance, cost, time, etc.
- Shortest path: the path of the least total weight
- A shortest path between any two vertices exists if the graph is connected
- Scenario: fastest route to travel from one location to another

# Dijkstra's Algorithm

- Proposed in 1956 by Edsger W. Dijkstra.
- Almost every GIS (Geographic Information System, or mapping software) uses a modification of Dijkstra's algorithm to provide directions
- Numerous versions of Dijkstra's Algorithm exist
- See original algorithm: DIJKSTRA, E. (1959). A Note on Two Problems in Connexion with Graphs. Numerische Mathematik, 1, 269-271.

# Dijkstra's Algorithm – notations

- Each vertex is given a two-part label

$$L(v) = (x, \omega(v))$$

- $x$: the predecessor of $v$ on the current best known path
- $\omega(v)$: the weight of the path that was used to get to $v$ from the designated starting vertex
- $F$: a set of vertices that are not highlighted yet

# Dijkstra's Algorithm – input and output

- **Input**: Weighted connected graph $G = (V, E)$ with nonnegative edge weights and vertices designated as *Start* and *End*
- **Output**: A shortest path from *Start* to *End* and its total weight $\omega(\mathit{End})$.

# Dijkstra's Algorithm – steps

1. For each vertex $v$ of $G$, assign a label $L(v)$:

$$L(v) = \begin{cases} (-, 0), & \text{if } v \text{ is } \textit{Start} \\ (-, \infty), & \text{Otherwise} \end{cases}$$

   Highlight *Start*. $F = V - \{\textit{Start}\}$. Let the *current vertex* be *Start*.

2. Update the labels for each vertex $v$ in $F$ that is a neighbor of *current vertex*, say $u$:

$$L(v) = \begin{cases} (u, \omega(u) + \omega(uv)), & \text{if } \omega(u) + \omega(uv) < \omega(v) \\ L(v), & \text{Otherwise} \end{cases}$$

3. Highlight a vertex $v$ in $F$ with the lowest weight and the edge indicated by its predecessor label. Remove $v$ from $F$. Redefine the *current vertex* to be $v$.

4. Repeat Steps 2 and 3 until the vertex *End* has been highlighted.

# Dijkstra's Algorithm – steps

5. The shortest path from *Start* to *End* is found by tracing back from *End* using the first component of the labels. The total weight of the path is the weight for *End* given in the second component of its label.

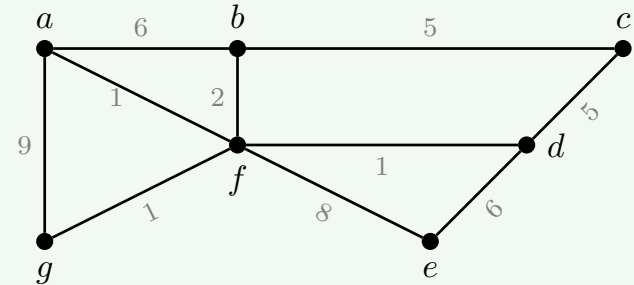
**Note**

- The set $F$ of vertices consists of all un-highlighted vertices and all are under consideration for becoming the next highlighted vertex
- It is important that we do not only consider the neighbors of the last vertex highlighted, as a path from a previously chosen vertex may in fact lead to the shortest path

# Dijkstra's Algorithm – example

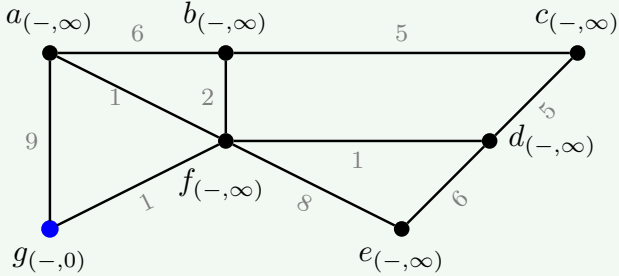
## Example

Take *Start* $= g$ and *End* $= c$.



# Dijkstra's Algorithm – example

## Example



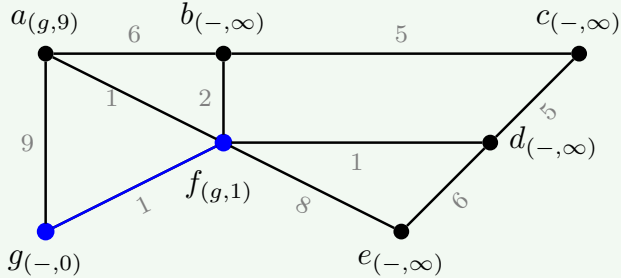

- Step 1. Highlight $g$. Label

$$L(v) = \begin{cases} (-,0) & v = g \\ (-,\infty) & \text{Otherwise} \end{cases}$$

$F = \{a, b, c, d, e, f\}$. The *current vertex* is $g$.

# Dijkstra's Algorithm – example

## Example



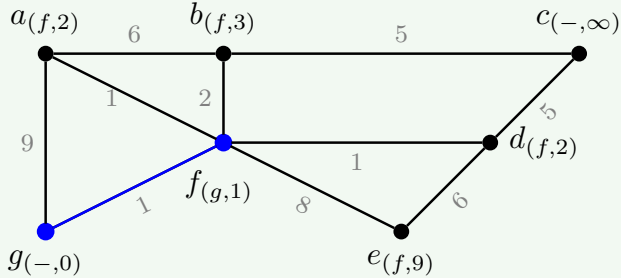

- Step 2. $F = \{a, b, c, d, e, f\}$. The neighbors of $g$ are $a$ and $f$.

$$\begin{aligned} \omega(g) + \omega(ga) &= 0 + 9 = 9 < \infty = \omega(a) \Longrightarrow L(a) = (g, 9) \\ \omega(g) + \omega(gf) &= 0 + 1 = 1 < \infty = \omega(f) \Longrightarrow L(f) = (g, 1) \end{aligned}$$

- Step 3. minimum weight: $f$; highlight $gf$ and $f$. $F = \{a, b, c, d, e\}$. *current vertex* is $f$.

# Dijkstra's Algorithm – example

## Example

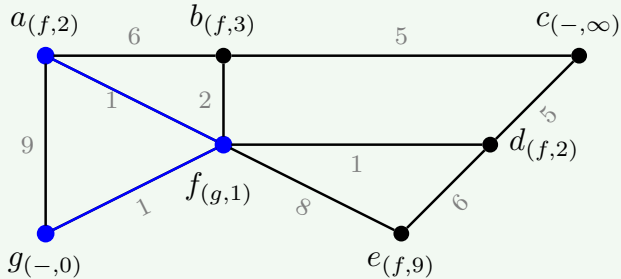



- Step 2. $F = \{a, b, c, d, e\}$. Neighbors of $f$: $a, b, d, e$

$$
\begin{aligned}
\omega(f) + \omega(fa) &= 1 + 1 = 2 < 9 = \omega(a) \Longrightarrow L(a) = (f, 2) \\
\omega(f) + \omega(fb) &= 1 + 2 = 3 < \infty = \omega(b) \Longrightarrow L(b) = (f, 3) \\
\omega(f) + \omega(fd) &= 1 + 1 = 2 < \infty = \omega(d) \Longrightarrow L(d) = (f, 2) \\
\omega(f) + \omega(fe) &= 1 + 8 = 9 < \infty = \omega(e) \Longrightarrow L(e) = (f, 9)
\end{aligned}
$$

# Dijkstra's Algorithm – example

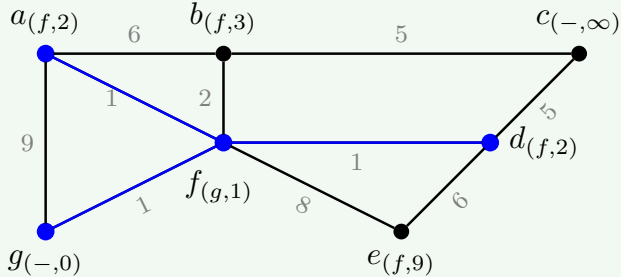
## Example




- Step 3. $F = \{a, b, c, d, e\}$. The minimum weight among the vertices in $F$ is attained by both $a$ and $d$. Choose one of them arbitrarily.
  - Let us highlight $fa$ and $a$.
  - $F = \{b, c, d, e\}$.
  - *current vertex* is $a$.

# Dijkstra's Algorithm – example

## Example




- Step 2. $F = \{b, c, d, e\}$. Neighbor of $a$: $b$

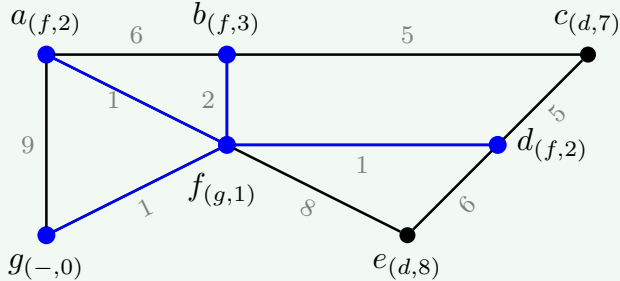
$$\omega(a) + \omega(ba) \quad = \quad 2 + 6 = 8 > 3 = \omega(b)$$

  We do not update the label of $b$.

- Step 3. Highlight $fd$ and $d$. $F = \{b, c, e\}$. *current vertex* is $d$.

# Dijkstra's Algorithm – example

## Example



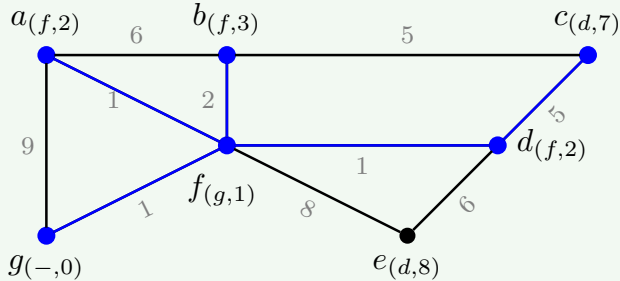

- Step 2. $F = \{b, c, e\}$. Neighbors of $d$: $c, e$

$$\begin{aligned} \omega(d) + \omega(dc) &= 2 + 5 = 7 < \infty = \omega(c) \Longrightarrow L(c) = (d, 7) \\ \omega(d) + \omega(de) &= 2 + 6 = 8 < 9 = \omega(e) \Longrightarrow L(e) = (d, 8) \end{aligned}$$

- Step 3. Highlight $fb$ and $b$. $F = \{c, e\}$. *current vertex* is $b$.

# Dijkstra's Algorithm – example

## Example




- Step 2. $F = \{c, e\}$. Neighbor of $b$: $c\ \omega(b) + \omega(bc) = 3 + 5 = 8 > 7 = \omega(c)$
- Step 3. Highlight $dc$ and $c$.
- Step 4. This terminates the iterations since we have reached *End*
- Step 5. The shortest path from $g$ to $c$ is $g \to f \to d \to c$, with total weight 7.

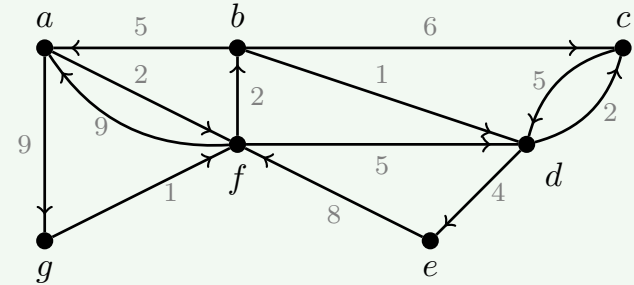
# Dijkstra's Algorithm for digraphs

- A digraph is a graph in which the edges now have a direction associated to them, which could be used to model a one-way street.
- For an arc $yx$, $x$ is the *head* and $y$ is the *tail*.
- In Step 2, instead of all neighbors, we consider only the *out-neighbors* of the current vertex, that is, vertices joined to it by outgoing arcs.

# Dijkstra's Algorithm for digraphs – example

## Example

*Start*= $g$, *End*= $c$




We record the changes in the following table.

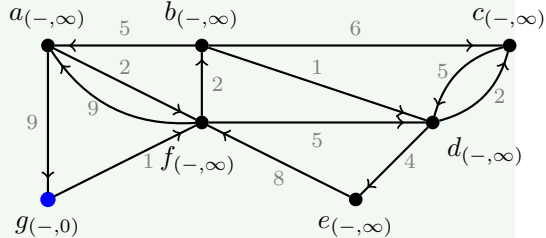
| $g$ | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ |
|---|---|---|---|---|---|---|

# Dijkstra's Algorithm for digraphs – example

## Example

- Step 1. *current vertex* is $g$,
  $F = \{a, b, c, d, e, f\}$

| $g$ | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ |
|---|---|---|---|---|---|---|
| 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |



# Dijkstra's Algorithm for digraphs – example

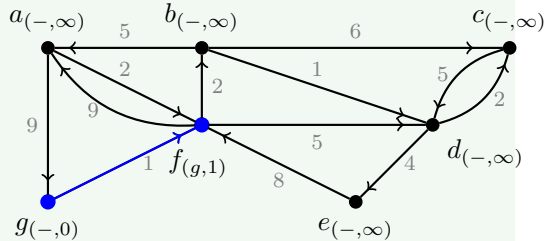
## Example

- Step 2. *current vertex* is $g$,
  $F = \{a, b, c, d, e, f\}$,
  out-neighbors of $g$: $f$

$$\omega(g) + \omega(gf) = 0 + 1 < \infty$$

- Step 3. *current vertex* is $f$,
  $F = \{a, b, c, d, e\}$

| | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ |
|---|---|---|---|---|---|---|
| | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $g(0)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $(g, 1)$ |

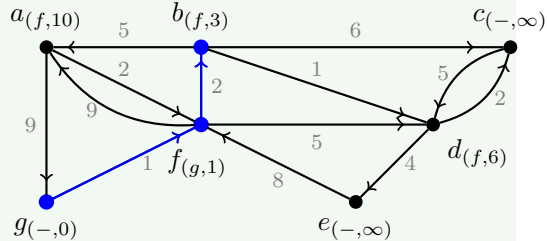


# Dijkstra's Algorithm for digraphs – example

## Example

- Step 2. *current vertex* is $f$,
  $F = \{a, b, c, d, e\}$,
  out-neighbors of $f$: $a, b, d$

$$\omega(f) + \omega(fa) = 1 + 9 = 10 < \infty,$$
$$\omega(f) + \omega(fb) = 1 + 2 = 3 < \infty,$$
$$\omega(f) + \omega(fd) = 1 + 5 = 6 < \infty.$$




- Step 3. *current vertex* is $b$,
  $F = \{a, c, d, e\}$

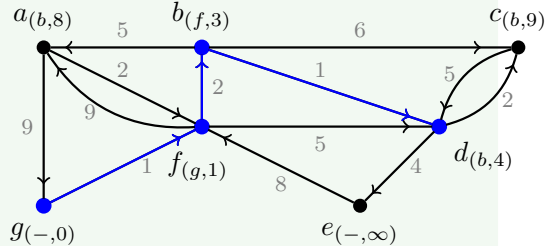
| | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ |
|---|---|---|---|---|---|---|
| | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $g(0)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $(g, 1)$ |
| $f(1)$ | $(f, 10)$ | $(f, 3)$ | $\infty$ | $(f, 6)$ | $\infty$ | $(g, 1)$ |

# Dijkstra's Algorithm for digraphs – example

## Example

- Step 2. *current vertex* is $b$,
  $F = \{a, c, d, e\}$,
  out-neighbors of $b$: $a, c, d$

$$\omega(b) + \omega(ba) = 3 + 5 = 8 < 10,$$
$$\omega(b) + \omega(bc) = 3 + 6 = 9 < \infty,$$
$$\omega(b) + \omega(bd) = 3 + 1 = 4 < 6.$$




- Step 3. *current vertex* is $d$, $F = \{a, c, e\}$

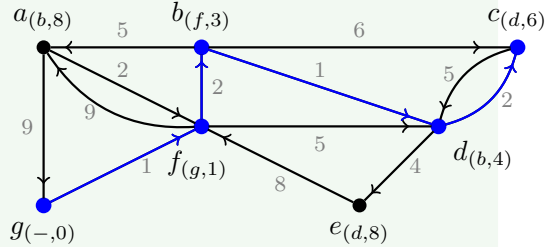
| | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ |
|---|---|---|---|---|---|---|
| | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $g(0)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $(g, 1)$ |
| $f(1)$ | $(f, 10)$ | $(f, 3)$ | $\infty$ | $(f, 6)$ | $\infty$ | $(g, 1)$ |
| $b(3)$ | $(b, 8)$ | $(f, 3)$ | $(b, 9)$ | $(b, 4)$ | $\infty$ | $(g, 1)$ |

# Dijkstra's Algorithm for digraphs – example

## Example

- Step 2. *current vertex* is $d$, $F = \{a, c, e\}$, out-neighbors of $d$: $c, e$

$$\omega(d) + \omega(dc) = 4 + 2 = 6 < 9,$$
$$\omega(d) + \omega(de) = 4 + 4 = 8 < \infty$$




- Step 3. *current vertex* is $c$
- Step 4. we have reached *End*
- Step 5. $g \to f \to b \to d \to c$, weight 6

| | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ |
|---|---|---|---|---|---|---|
| | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $g(0)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $(g,1)$ |
| $f(1)$ | $(f,10)$ | $(f,3)$ | $\infty$ | $(f,6)$ | $\infty$ | $(g,1)$ |
| $b(3)$ | $(b,8)$ | $(f,3)$ | $(b,9)$ | $(b,4)$ | $\infty$ | $(g,1)$ |
| $d(4)$ | $(b,8)$ | $(f,3)$ | $(d,6)$ | $(b,4)$ | $(d,8)$ | $(g,1)$ |

# Remark

- It is possible for a path not to exist from one vertex to another based upon the direction of the arcs
- For example, if $a$ is the head of every arc incident to $a$, then no directed path of positive length can start at $a$.
- For digraphs, if all unhighlighted vertices have weight $\infty$ before *End* is reached, the algorithm terminates (Step 3) and reports that no directed path from *Start* to *End* exists.

# Shortest Paths, Project Scheduling, Trees, and Networks

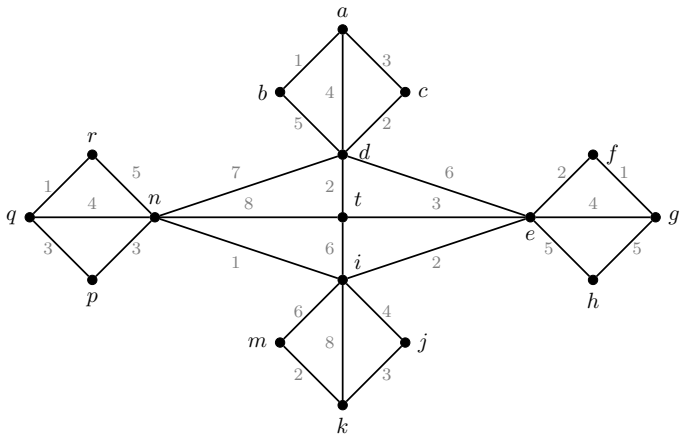
- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Chinese Postman Problem

- Recall: for a connected, undirected graph with nonnegative edge weights, the *Chinese Postman Problem* is to find a closed walk of minimum total weight that traverses every edge at least once.
- First, Eulerize the graph by pairing odd-degree vertices and duplicating shortest paths between them, minimizing the total added weight.
- Then, find an Eulerian circuit in the resulting graph.
- We now use Dijkstra's Algorithm to find the shortest paths needed for an optimal Eulerization.

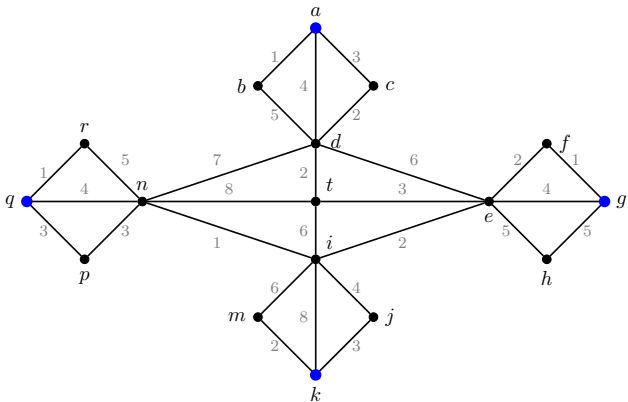
# Chinese Postman Problem – example

Use Dijkstra's Algorithm to determine the best pairing of the odd vertices and the total weight of the duplicated edges in an Eulerization of the graph.



# Chinese Postman Problem – example

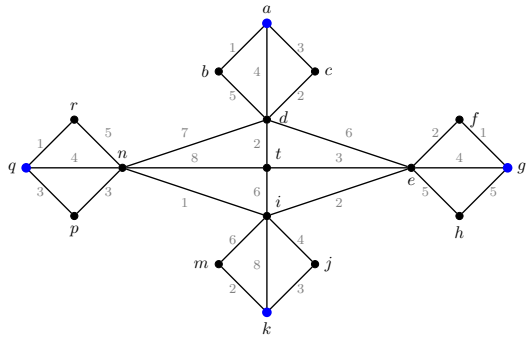
- Four odd vertices: $a, g, k, q$
- Three possible pairings $a-g, k-q$, or $a-k, g-q$, or $a-q, g-k$



# Chinese Postman Problem – example

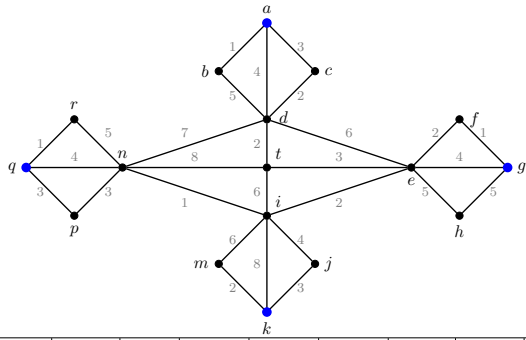
- Three possible pairings $a-g, k-q$, or $a-k, g-q$, or $a-q, g-k$
- Applying Dijkstra's Algorithm, we find the shortest path for each paired set of vertices, and hence the total weight of the duplicated paths needed to Eulerize the graph.

# Chinese Postman Problem – example




| | $b$ | $c$ | $d$ | $n$ | $e$ | $t$ | $r$ | $q$ | $p$ | $f$ | $g$ | $h$ | $i$ | $m$ | $j$ | $k$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $a(0)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $b(1)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $c(3)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |

# Chinese Postman Problem – example




| | $b$ | $c$ | $d$ | $n$ | $e$ | $t$ | $r$ | $q$ | $p$ | $f$ | $g$ | $h$ | $i$ | $m$ | $j$ | $k$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $a(0)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $b(1)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $c(3)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $d(4)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(d,10)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $t(6)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $(t,12)$ | $\infty$ | $\infty$ | $\infty$ |
| $e(9)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $(e,11)$ | $(e,13)$ | $(e,14)$ | $(e,11)$ | $\infty$ | $\infty$ | $\infty$ |
| $f(11)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $(e,11)$ | $(f,12)$ | $(e,14)$ | $(e,11)$ | $\infty$ | $\infty$ | $\infty$ |
| $i(11)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $(e,11)$ | $(f,12)$ | $(e,14)$ | $(e,11)$ | $(i,17)$ | $(i,15)$ | $(i,19)$ |
| $n(11)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $(n,16)$ | $(n,15)$ | $(n,14)$ | $(e,11)$ | $(f,12)$ | $(e,14)$ | $(e,11)$ | $(i,17)$ | $(i,15)$ | $(i,19)$ |

# Chinese Postman Problem – example

| | $b$ | $c$ | $d$ | $n$ | $e$ | $t$ | $r$ | $q$ | $p$ | $f$ | $g$ | $h$ | $i$ | $m$ | $j$ | $k$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $a(0)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $b(1)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $c(3)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $d(4)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(d,10)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $t(6)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $(t,12)$ | $\infty$ | $\infty$ | $\infty$ |
| $e(9)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $(e,11)$ | $(e,13)$ | $(e,14)$ | $(e,11)$ | $\infty$ | $\infty$ | $\infty$ |
| $f(11)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $(e,11)$ | $(f,12)$ | $(e,14)$ | $(e,11)$ | $\infty$ | $\infty$ | $\infty$ |
| $i(11)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $\infty$ | $\infty$ | $\infty$ | $(e,11)$ | $(f,12)$ | $(e,14)$ | $(e,11)$ | $(i,17)$ | $(i,15)$ | $(i,19)$ |
| $n(11)$ | $(a,1)$ | $(a,3)$ | $(a,4)$ | $(d,11)$ | $(t,9)$ | $(d,6)$ | $(n,16)$ | $(n,15)$ | $(n,14)$ | $(e,11)$ | $(f,12)$ | $(e,14)$ | $(e,11)$ | $(i,17)$ | $(i,15)$ | $(i,19)$ |

$$a \to d \to t \to e \to f \to g$$

with total weight $12$

# Chinese Postman Problem – example

- Three possible pairings $a-g, k-q$, or $a-k, g-q$, or $a-q, g-k$
- Applying Dijkstra's Algorithm, we find the shortest path for each paired set of vertices, and hence the total weight of the duplicated paths needed to Eulerize the graph.

| **Path Pairs** | **Weight** | **Total Weight** |
|---|---|---|
| $a \to d \to t \to e \to f \to g$ | 12 | |
| $k \to j \to i \to n \to q$ | 12 | 24 |
| $a \to d \to t \to e \to i \to j \to k$ | 18 | |
| $g \to f \to e \to i \to n \to q$ | 10 | 28 |
| $a \to d \to n \to q$ | 15 | |
| $g \to f \to e \to i \to j \to k$ | 12 | 27 |

Exercise: verify these shortest paths and total weights.

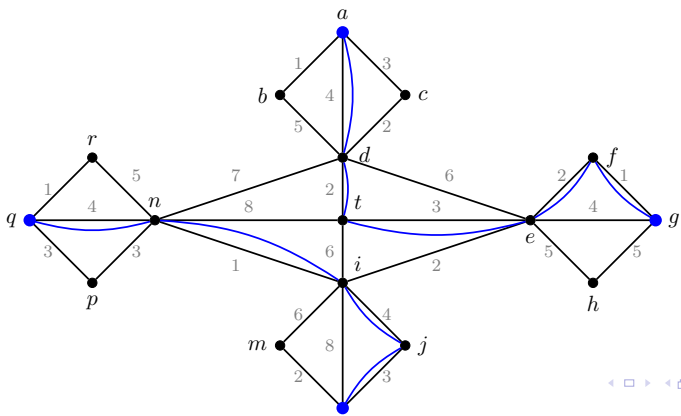
# Chinese Postman Problem – example

- Once the shortest paths have been found, we duplicate the edges along those paths to obtain an Eulerization.
- However, the paths in a pairing may share edges.
- For example, both paths in the second pairing contain the edge $ei$.
- We can omit both copies of $ei$ and retain the other duplicated edges.
- Removing both copies preserves the parity of every vertex degree and reduces the total added weight by $2\omega\,(ei) = 4$, making this Eulerization equal in cost to the first one.

| | | |
|---|---|---|
| $adteijk$ | 18 | |
| $gfeinq$ | 10 | 28 |

# Chinese Postman Problem – example

- duplicate the edges along $a \to d \to t \to e \to f \to g$ and $k \to j \to i \to n \to q$.
- duplicate the edges along $a \to d \to t \to e \to i \to j \to k$ and $g \to f \to e \to i \to n \to q$, except for the shared edge $ei$, which is omitted from both paths.
- Both choices give an Eulerization with total added weight $24$.



# Remarks

- The use of Dijkstra's Algorithm allows for a methodical approach to the Chinese Postman Problem.
- However, even for small examples, the number of shortest paths that must be considered can be quite large.
- A more efficient approach uses matchings.

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Definitions

## Definition

Consider a project consisting of multiple tasks.

- *Task*: a required step of a project that cannot be broken into smaller pieces. Labeled with lowercase letters
- *Processor*: the unit (such as a person) that completes a task. Labeled as $P_1, P_2$, etc. At any time, a processor is either idle or busy performing a task.
- At any stage of a project, a task can be in one of four states:
  - *eligible*: the task can be performed
  - *ineligible*: the task cannot be performed
  - *in execution*: the task is currently being performed
  - *completed*: the task has been completed

  A task is eligible when all tasks on which it depends have been completed.

# Definitions

## Definition

Consider a project containing multiple parts of steps.

- *Processing time* of a task: the time it takes to complete the task, denoted by $\mathrm{pt}(v)$ for task $v$
- *Precedence relationship*: task $b$ relies on the completion of task $a$ before it can be eligible
- *Finishing time* of a schedule is the total time used in that schedule
- *Optimal time* of a project is the minimum finishing time among all possible schedules, denoted OPT.

# Tasks and schedules – example

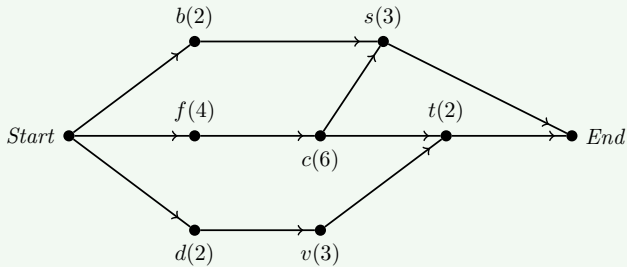
**Example**

- Party planning

| **Task** | Vertex Name | Processing Time | Precedence Relationships |
|---|---|---|---|
| **Buy Food** | $f$ | 40 | |
| **Buy Drinks** | $b$ | 20 | |
| **Dust** | $d$ | 20 | |
| **Vacuum** | $v$ | 30 | $d$ |
| **Cook Food** | $c$ | 60 | $f$ |
| **Set Out Drinks** | $s$ | 30 | $b, c$ |
| **Set Table** | $t$ | 20 | $v, c$ |

# Tasks and schedules – example

## Example




- It is customary to include vertices representing the start and end of the project, and to place vertices so as to avoid edge crossings whenever possible.
- The processing times are shown in parentheses next to the vertex labels
- Edges: precedence relationships

# Priority List Model

- Once a digraph has been created, the next step is to determine which processor (or person) should complete each task
- This may be easy for a project with only a few tasks, or when the dependencies between tasks are not complex.
- As complexity grows, we will need a procedure for assigning tasks
- *Priority List Model*:
  - Whenever a processor becomes available, assign the highest-priority eligible task that has not yet been assigned.
  - Precedence relationships (displayed in the digraph) determine eligibility. Ineligible tasks are skipped until all their predecessors have been completed.

# Priority List Model – example

## Example

- Continuing from the previous example
- Take priority list: $b - d - t - v - s - f - c$
- Consider two processors
- Each step represents a moment in time where a decision must be made

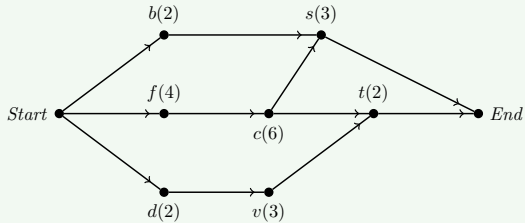
| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 | 140 | 150 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $b$ | $b$ | | | | | | | | | | | | | |
| $P_2$ | $d$ | $d$ | | | | | | | | | | | | | |

- Step 1. $T = 0$ The first item in the priority list is $b$, since $b$ does not rely on any other task, assign it to $P_1$. The next item $d$ is also eligible. Assign $d$ to $P_2$.

# Priority List Model – example

## Example

Priority list: $b - d - t - v - s - f - c$



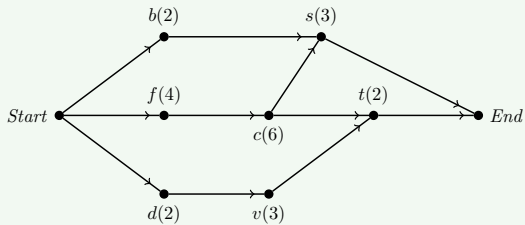

| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 | 140 | 150 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $b$ | $b$ | $v$ | $v$ | $v$ | | | | | | | | | | |
| $P_2$ | $d$ | $d$ | $f$ | $f$ | $f$ | $f$ | | | | | | | | | |

- Step 2. $T = 20$. The next point at which a processor is free to pick up a task is at $20$ minutes. The next task on the list is $t$, but it is ineligible. Both $v$ and $f$ are eligible.

# Priority List Model – example

## Example

Priority list: $b - d - t - v - s - f - c$



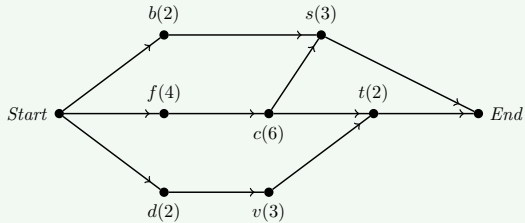

| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 | 140 | 150 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $b$ | $b$ | $v$ | $v$ | $v$ | $*$ | | | | | | | | | |
| $P_2$ | $d$ | $d$ | $f$ | $f$ | $f$ | $f$ | | | | | | | | | |

- Step 3. $T = 50$. At $50$ minutes, Processor $1$ is ready for a new task. All tasks remaining require $f$ to be complete. $P_1$ will remain idle until $f$ is complete

# Priority List Model – example

## Example

Priority list: $b - d - t - v - s - f - c$



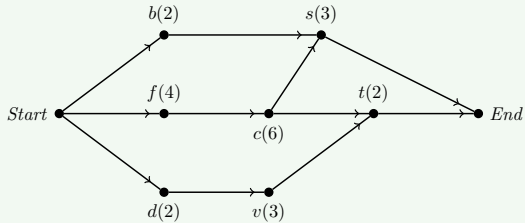

| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 | 140 | 150 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $b$ | $b$ | $v$ | $v$ | $v$ | $*$ | $c$ | $c$ | $c$ | $c$ | $c$ | $c$ | | | |
| $P_2$ | $d$ | $d$ | $f$ | $f$ | $f$ | $f$ | $*$ | $*$ | $*$ | $*$ | $*$ | $*$ | | | |

- Step 4. $T = 60$. At $60$ minutes, both processors are ready to take up a new task. The only eligible task is $c$. By convention, we assign the task to the lower indexed processor. The other processor remains idle

# Priority List Model – example

## Example

Priority list: $b - d - t - v - s - f - c$




| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 | 140 | 150 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $b$ | $b$ | $v$ | $v$ | $v$ | $*$ | $c$ | $c$ | $c$ | $c$ | $c$ | $c$ | $t$ | $t$ | $*$ |
| $P_2$ | $d$ | $d$ | $f$ | $f$ | $f$ | $f$ | $*$ | $*$ | $*$ | $*$ | $*$ | $*$ | $s$ | $s$ | $s$ |

- Step 5. $T = 120$. Once $c$ is complete, we can assign $t$ to $P_1$ and $s$ to $P_2$

The priority list $b - d - t - v - s - f - c$ yields a finishing time of $150$ minutes using two processors.

# Remarks

- The schedule we obtained contains a large amount of idle time, $80$ minutes in total
- Although some idle time may be unavoidable, its presence should indicate that more investigation is warranted.
- The priority list given did not seem to have any connection with the digraph (in fact, it was generated randomly)
- A better approach would be to use information from the digraph to obtain a good priority list.

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Critical path

- *Critical path*: a path from *Start* to *End* with the greatest total processing time.
- This path is of interest because it easily identifies restrictions on the completion time of a project
- In addition, it indicates which tasks should be prioritized.
- To find the critical path, we first need to find the *critical times* of all vertices in the graph.

# Critical time

**Definition**

The *critical time* $\mathrm{ct}[x]$ of a vertex $x$ is the processing time of $x$ plus the maximum critical time among all out-neighbors $y$ of $x$, with $\mathrm{ct}[\mathit{End}] = 0$. For any vertex $x \neq \mathit{End}$,

$$\mathrm{ct}[x] = \mathrm{pt}(x) + \max\{\, \mathrm{ct}[y] \mid xy \text{ is an arc} \,\}$$

**Note**

In the definition, $y$ is an out-neighbor of $x$

# Critical Path Algorithm

**Input**: A directed acyclic project graph $G$ with processing times given, where every task lies on a directed path from *Start* to *End*

**Steps**:

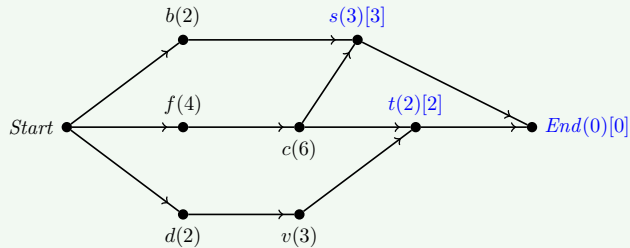
1. Label the vertex *End* with $\mathrm{pt}(\textit{End}) = 0$ and $\mathrm{ct}[\textit{End}] = 0$. For any vertex $x$ with an arc to *End*, define $\mathrm{ct}[x] = \mathrm{pt}(x)$.
2. Process the vertices in reverse topological order, starting from *End*. Calculate the critical time of each vertex only after the critical times of all its out-neighbors have been determined.
3. Once all critical times have been obtained, find a path from *Start* to *End* by always following, from the current vertex, an outgoing arc to a neighbor of largest critical time.
4. Create a priority list by ordering vertices by decreasing critical time

**Output**: Critical path and critical path priority list

# Critical Path Algorithm – example

## Example

- Continuing from the previous example
- We will use brackets for the critical times, distinguishing them from the processing times

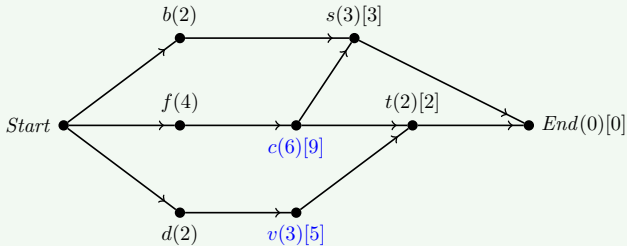



- Step 1. We express processing times in units of $10$ minutes. Label *End* with critical time $0$. Since $s$ and $t$ have arcs to *End*, set

$$\mathrm{ct}[s] = \mathrm{pt}(s) = 3, \quad \mathrm{ct}[t] = \mathrm{pt}(t) = 2$$

# Critical Path Algorithm – example

## Example




- Step 2. As $v$ has a single arc to $t$

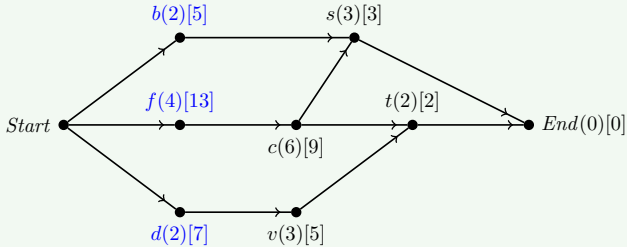
$$\mathrm{ct}[v] = \mathrm{pt}(v) + \mathrm{ct}[t] = 3 + 2 = 5$$

$c$ has an arc to both $s$ and $t$. $\mathrm{ct}[s] > \mathrm{ct}[t]$

$$\mathrm{ct}[c] = \mathrm{pt}(c) + \mathrm{ct}[s] = 6 + 3 = 9$$

# Critical Path Algorithm – example

## Example

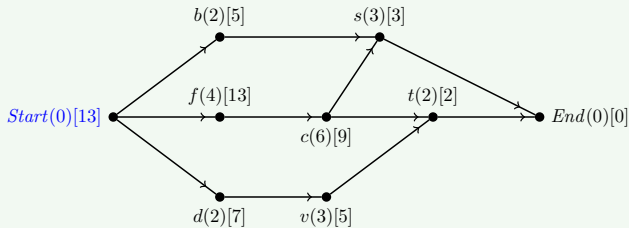



- Step 3. The remaining vertices each have a single arc to previously considered vertices

$$\mathrm{ct}[b] = \mathrm{pt}(b) + \mathrm{ct}[s] = 2 + 3 = 5$$
$$\mathrm{ct}[f] = \mathrm{pt}(f) + \mathrm{ct}[c] = 4 + 9 = 13$$
$$\mathrm{ct}[d] = \mathrm{pt}(d) + \mathrm{ct}[v] = 2 + 5 = 7$$

# Critical Path Algorithm – example

## Example

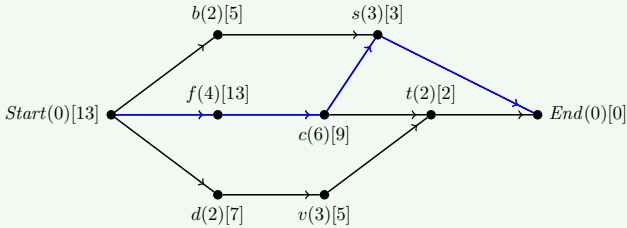



- Step 4. Label the processing time of *Start* as $0$. $f$ is the out-neighbor with the largest critical time

$$\mathrm{ct}[\mathit{Start}] = 0 + 13 = 13$$

# Critical Path Algorithm – example

## Example




- Step 5. Follow the path from *Start* to *End* where the vertices are chosen based on the largest critical times. This gives the path

$$Start \to f \to c \to s \to End$$

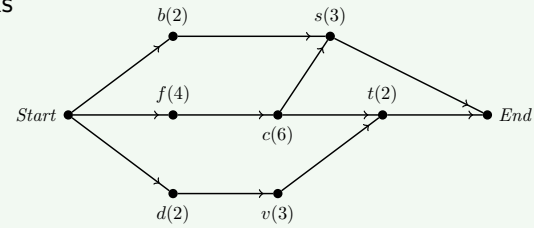
with total time $130$. Ordering the vertices by decreasing critical time gives the critical path priority list

$$f - c - d - b - v - s - t$$

# Priority List Model for project scheduling – example

## Example

Now we use the critical path priority list $f - c - d - b - v - s - t$ to find a new schedule for the tasks



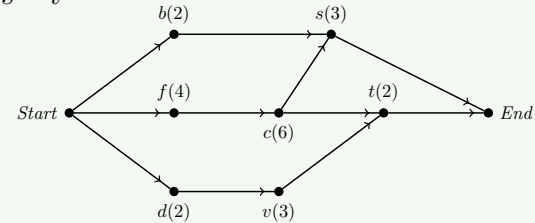

| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $f$ | $f$ | $f$ | $f$ | | | | | | | | | |
| $P_2$ | $d$ | $d$ | | | | | | | | | | | |

- Step 1. $T = 0$. Since $f$ is the first item in the list, assign it to $P_1$. $c$ is not eligible. Assign $d$ to $P_2$

# Priority List Model for project scheduling – example

## Example

$f - c - d - b - v - s - t$



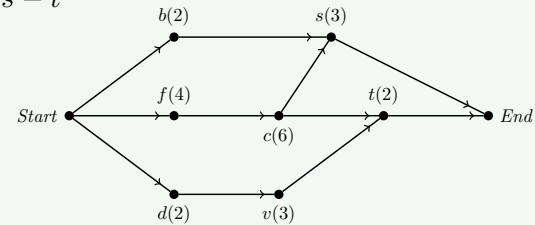

| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $f$ | $f$ | $f$ | $f$ | | | | | | | | | |
| $P_2$ | $d$ | $d$ | $b$ | $b$ | | | | | | | | | |

- Step 2. $T = 20$. $P_2$ can be assigned a new task. Assign $b$, the next eligible task to $P_2$

# Priority List Model for project scheduling – example

## Example

$f - c - d - b - v - s - t$



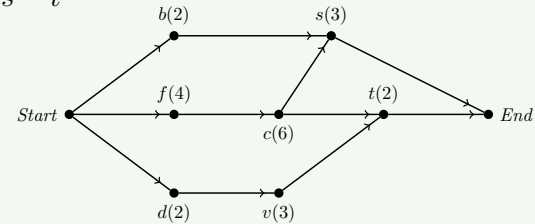

| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $f$ | $f$ | $f$ | $f$ | $c$ | $c$ | $c$ | $c$ | $c$ | $c$ | | | |
| $P_2$ | $d$ | $d$ | $b$ | $b$ | $v$ | $v$ | $v$ | | | | | | |

- Step 3. $T = 40$. $c$ is eligible. The next eligible task is $v$

# Priority List Model for project scheduling – example

## Example

$f - c - d - b - v - s - t$



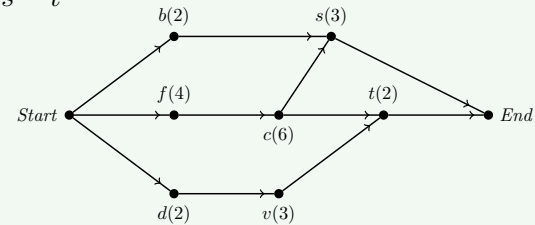

| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $f$ | $f$ | $f$ | $f$ | $c$ | $c$ | $c$ | $c$ | $c$ | $c$ | | | |
| $P_2$ | $d$ | $d$ | $b$ | $b$ | $v$ | $v$ | $v$ | $*$ | $*$ | $*$ | | | |

- Step 4. $T = 70$. All remaining tasks are ineligible since they rely on the completion of $c$. $P_2$ remains idle.

# Priority List Model for project scheduling – example

## Example

$f - c - d - b - v - s - t$




| | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 | 110 | 120 | 130 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $f$ | $f$ | $f$ | $f$ | $c$ | $c$ | $c$ | $c$ | $c$ | $c$ | $s$ | $s$ | $s$ |
| $P_2$ | $d$ | $d$ | $b$ | $b$ | $v$ | $v$ | $v$ | $*$ | $*$ | $*$ | $t$ | $t$ | $*$ |

- Step 5. $T = 100$. $s$ and $t$ are now eligible
- Finishing time: $130$ minutes, $40$ minutes of idle time

# Remarks

- Both schedules contained some idle time, though the one using the critical path priority list had only half as much as the initial example. This is partly because tasks on the critical path were prioritized.
- The schedule above must be optimal since its finishing time is equal to the critical time of Start
- In general, the critical path priority list results in a very good, though not always optimal, schedule

# Optimal schedule

- The optimal time of a schedule is no less than the critical time of *Start*

$$OPT \geq \mathrm{ct}[\mathit{Start}]$$

- Calculate the sum of all processing times of all tasks. The optimal time is no less than this sum divided by the total number of processors used

$$OPT \geq \frac{\sum_v \mathrm{pt}(v)}{\text{number of processors}}$$

# Optimal schedule – example

## Example

With our running example, sum of all processing times is $220$ minutes

- Using two processors $OPT \geq \frac{220}{2} = 110$
- Using three processors $OPT \geq \frac{220}{3} \approx 73.33$
- $\mathrm{ct}[\mathit{Start}] = 130$. Neither of the above two calculations provides additional insight into the optimal schedule.

## Remark

The schedule constructed earlier shows that two processors are sufficient to achieve the optimal finishing time of $130$ minutes, since this equals the critical-path lower bound. Consequently, additional processors cannot reduce the optimal finishing time.

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Definition

## Definition

A graph $G$ is

- *acyclic* if there are no cycles or circuits in the graph
- a *network* if it is connected
- a *tree* if it is an acyclic network, i.e. acyclic and connected
- a *forest* if it is an acyclic graph

A vertex of degree $1$ is called a *leaf*.

# Trees – example

## Example

Game

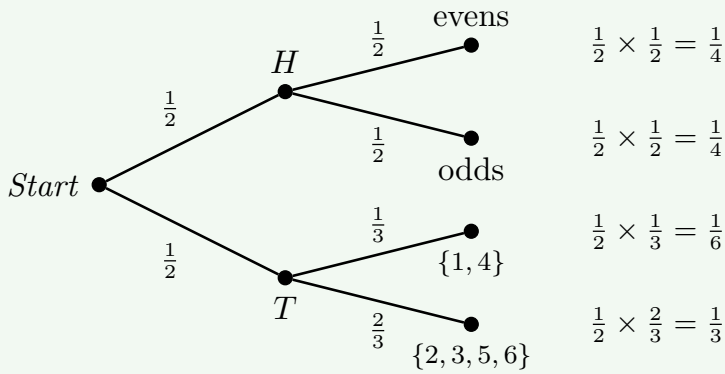
- Adam flips a coin, Jano rolls a die
- Adam gets heads, Jano rolls an even number $\rightarrow$ Jano wins $2$ Euros
- Adam gets heads, Jano rolls an odd number $\rightarrow$ Adam wins $3$ Euros
- Adam gets tails, Jano rolls $1$ or $4$ $\rightarrow$ Jano wins $5$ Euros
- Adam gets tails, Jano rolls $2, 3, 5$, or $6$ $\rightarrow$ Adam wins $2$ Euros

A probability tree – vertices representing possible outcomes, edges labeled with the probability of the outcome

# Trees – example

## Example

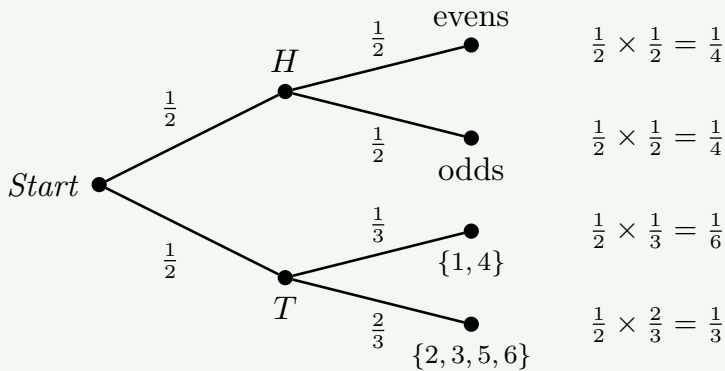
- Adam gets heads, Jano rolls an even number $\rightarrow$ Jano wins $2$ Euros
- Adam gets heads, Jano rolls an odd number $\rightarrow$ Adam wins $3$ Euros
- Adam gets tails, Jano rolls $1$ or $4$ $\rightarrow$ Jano wins $5$ Euros
- Adam gets tails, Jano rolls $2, 3, 5$, or $6$ $\rightarrow$ Adam wins $2$ Euros



# Trees – example

## Example

- The probability Jano wins $5$ Euros (tails and $1$ or $4$) is $\frac{1}{6}$
- The probability that Jano wins any money is $\frac{1}{6} + \frac{1}{4} = \frac{5}{12}$



# Trees – example

## Example

- Trees can be used to store information for quick access
- Consider the following sequence of numbers

$$4,\ 2,\ 7,\ 10,\ 1,\ 3, 5$$

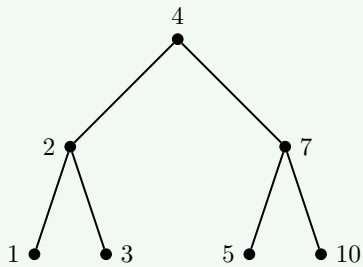
- We can form a tree by creating a vertex for each number in the list
- To insert each new number into the binary search tree, start at the root. Compare it with the current vertex and move left if it is smaller or right if it is greater. Repeat until an empty position is reached, and insert the new number there.
- If we add the restriction that no vertex can have more than two edges coming down from it, then we are forming a binary tree.

# Trees – example

## Example

- To insert each new number into the binary search tree, start at the root. Compare it with the current vertex and move left if it is smaller or right if it is greater. Repeat until an empty position is reached, and insert the new number there.
- Each vertex has at most two children.

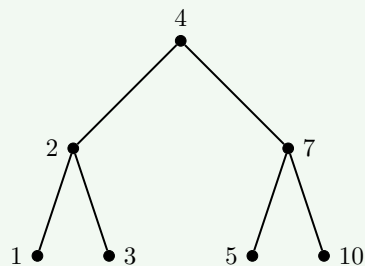
$$4,\ 2,\ 7,\ 10,\ 1,\ 3,\ 5$$



# Trees – example

## Example

- If we want to search for an item, we may need far fewer comparisons than in a linear search.
- Find item $5$: first compare it to $4$ and move right, then compare it to $7$ and move left, and finally compare it to $5$.
- Only three comparisons are needed to find the item.



# Properties of trees

1. For every $n \geq 1$, any tree with $n$ vertices has $n - 1$ edges
2. For any tree with $n \geq 1$ vertices, the sum of the degrees is $2n - 2$
3. Every tree with at least two vertices contains at least two leaves
4. Any network on $n$ vertices with $n - 1$ edges must be a tree
5. For any two vertices in a tree, there is a unique path between them
6. The removal of any edge of a tree will disconnect the graph

Note

- $1 \Rightarrow 2$
- By counting the degrees of vertices, $2 \Rightarrow 3$

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Spanning Tree

## Definition

- A *subgraph* $H$ of a graph $G$ is a graph such that $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$.
- $H$ is a *spanning subgraph* if $V(H) = V(G)$
- A *spanning tree* is a spanning subgraph that is also a tree.

## Note

- If an edge appears in a subgraph, then both endpoints must also be included in the subgraph
- If a vertex appears in a subgraph, any number of its incident edges may be included

# Kruskal's Algorithm

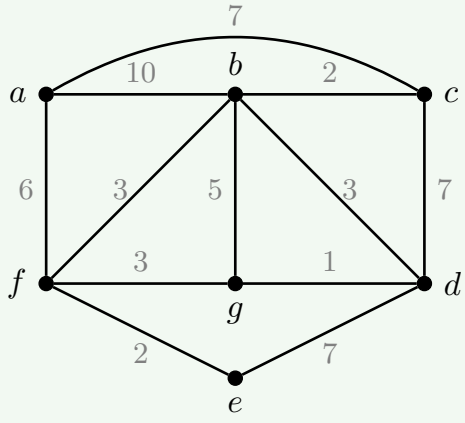
- First published in $1956$ by Joseph Kruskal, an American mathematician
- Input: weighted connected graph $G = (V, E, \omega)$
- Steps
  1. Among all unhighlighted edges whose addition would not create a cycle, choose one of minimum weight. Highlight it and add it to $T = (V, E', \omega')$.
  2. Repeat Step 1 until $T$ contains $|V| - 1$ edges.
- Output: minimum spanning tree (MST) $T$ of $G$

**Note**

- If several eligible edges have the same minimum weight, choose one arbitrarily.
- At each step of the algorithm we are building a forest subgraph that will eventually result in a spanning tree

# Kruskal's Algorithm – example

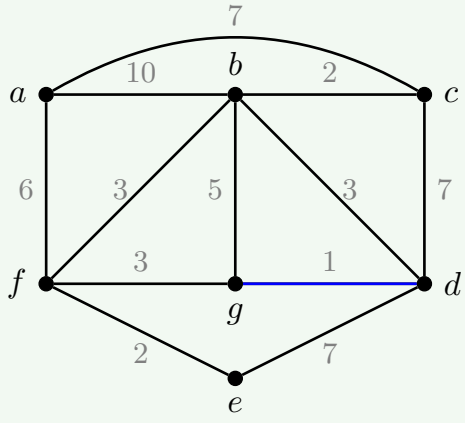
## Example




- Step 1. edge with the least weight: $gd$

# Kruskal's Algorithm – example

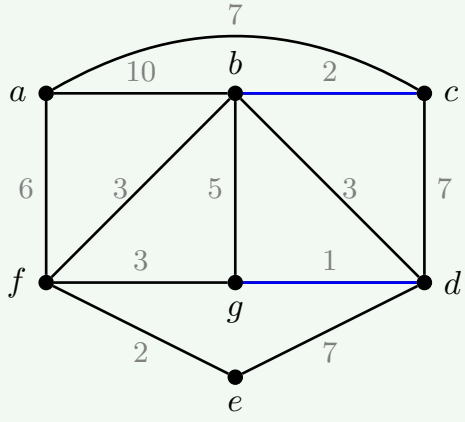
## Example




- Step 1. two choices: $ef$ and $bc$, let's choose $bc$

# Kruskal's Algorithm – example

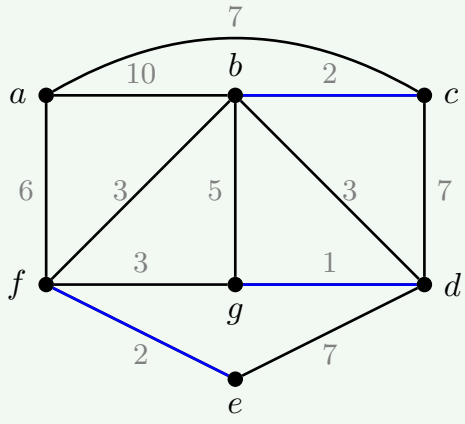
## Example




- Step 1. the other edge with weight 2 is still a valid choice. Highlight $ef$

# Kruskal's Algorithm – example

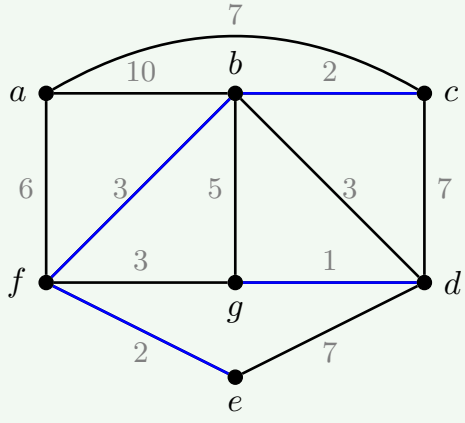
## Example




- Step 1. the next smallest edge weight is $3$. We randomly pick $bf$

# Kruskal's Algorithm – example

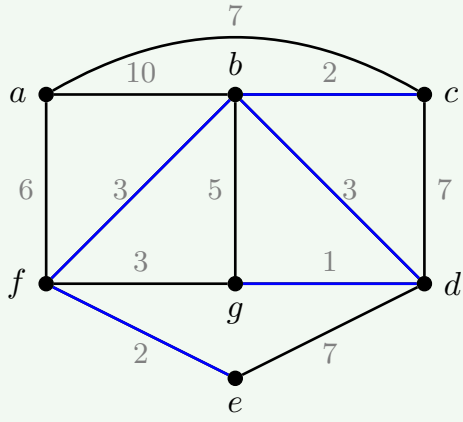
Example




- Step 1. both of the other edges of weight $3$ are still available. We choose $bd$

# Kruskal's Algorithm – example

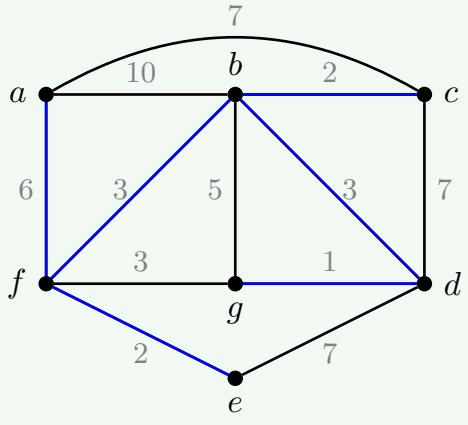
## Example




- Step 1. cannot choose $fg$. The next smallest edge weight is $5$. But we cannot choose $bg$. Next available edge is $af$ of weight $6$

# Kruskal's Algorithm – example

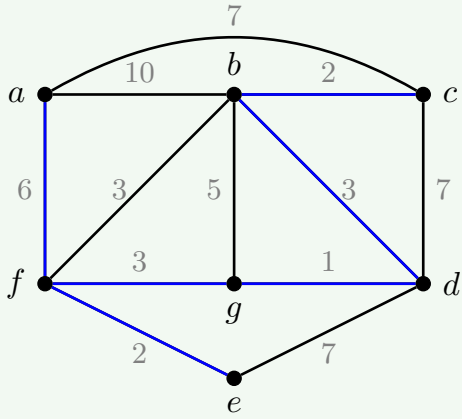
## Example

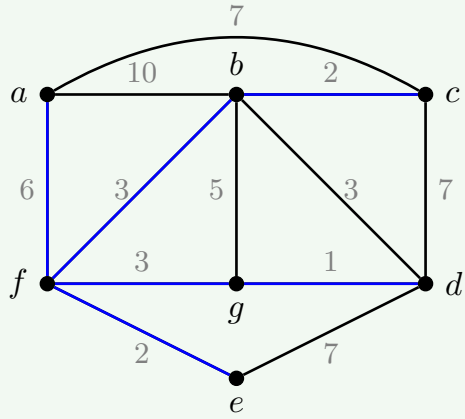



- Step 2. We now have a connected graph containing all 7 vertices and 6 edges, with no cycles. Hence, it is a tree, and we can stop. The MST has total weight 17.

# Kruskal's Algorithm – example

## Example




- When we were choosing an edge of weight $3$, we could have made a different choice.

# Kruskal's Algorithm – remarks

- When we skipped over an edge, e.g. $bg$ of weight $5$, we did so because including it would create a cycle
- This means a path between the endpoints of that edge, e.g. $b$ and $g$, must already exist and the other edges along that path must each be of weight no greater than the edge we skip over
- When finding a minimum spanning tree, an edge that is strictly heavier than every other edge in a cycle cannot belong to any minimum spanning tree. If several edges share the maximum weight, some of them may belong to a minimum spanning tree.

# Prim's Algorithm

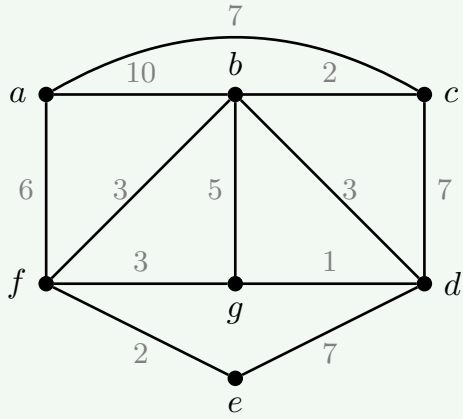
- Named after Robert C. Prim, American mathematician and computer scientist, published in 1957
- Originally discovered by Vojtěch Jarnik, Czech mathematician, in 1930
- Root: a clear starting point for the tree

# Prim's Algorithm

- Input: Weighted connected graph $G = (V, E)$
- Steps
  1. Let $v$ be the root. If no root is specified, choose a vertex at random. Highlight it and add it to $T = (V', E')$
  2. Among all edges incident to $v$, choose the one of minimum weight. Highlight it. Add the edge and its other endpoint to $T$.
  3. Let $S$ be the set of all edges with exactly one endpoint in $V(T)$. Choose an edge of minimum weight from $S$, highlight it, and add this edge and its endpoint outside $V(T)$ to $T$.
  4. Repeat Step 3 until $T$ contains all vertices of $G$.
- Output: rooted MST $T$ of $G$

# Prim's Algorithm – example

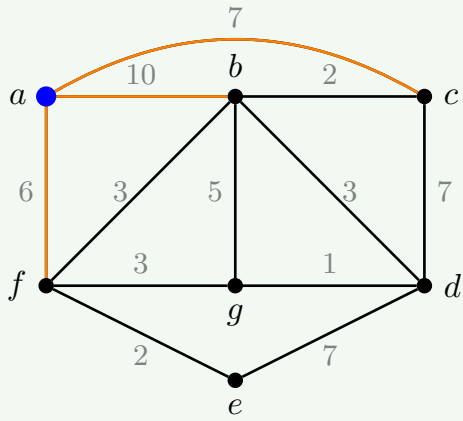
## Example




- Same example as before
- Step 1. Choose $a$ as the starting vertex

# Prim's Algorithm – example

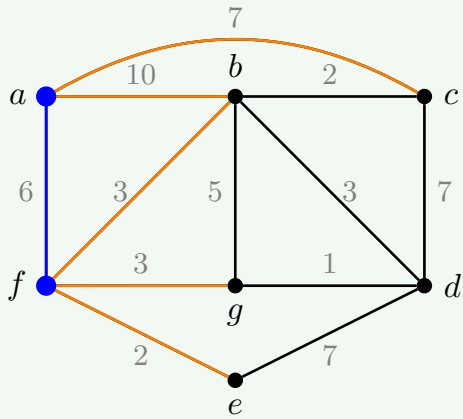
## Example




- Step 2. among the edges incident to $a$, $af$, $ab$, $ac$, the edge of the least weight is $af$

# Prim's Algorithm – example

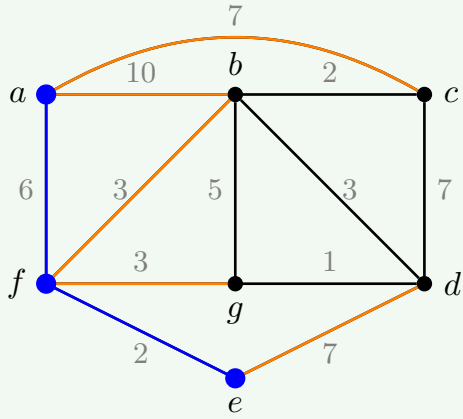
## Example




- Step 3. $S$ consists of edges with exactly one endpoint in the current tree, whose vertices are $a$ and $f$. The edge with the minimum weight is $ef$

# Prim's Algorithm – example

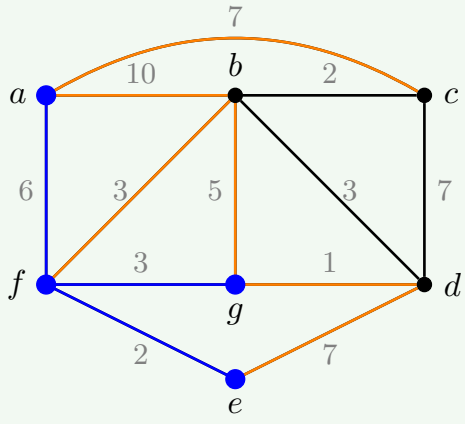
Example




- Step 3. $S$ consists of edges with exactly one endpoint in the current tree, whose vertices are $a$, $e$, and $f$. The minimum-weight edges are $fb$ and $fg$. Let us choose

# Prim's Algorithm – example

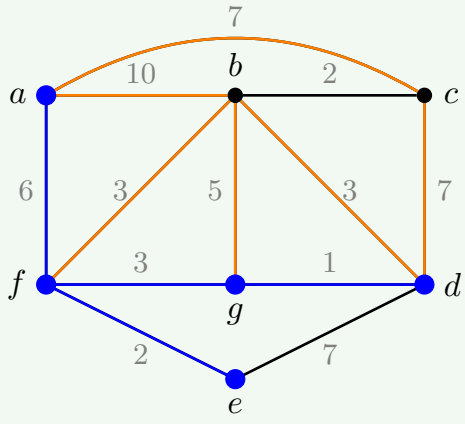
## Example




- Step 3. The next edge to add to the tree is $dg$

# Prim's Algorithm – example

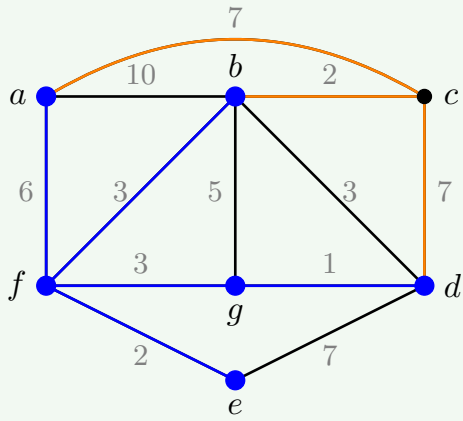
## Example




- Step 3. There are two possible minimum weight edges, $bf$ or $bd$. We choose $bf$

# Prim's Algorithm – example

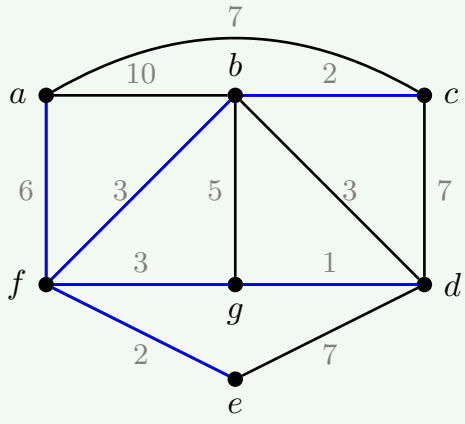
## Example




- Step 3. Now the only edges to consider are those with one endpoint of $c$ since this is the only vertex not part of our tree. The edge of minimum weight is $bc$

# Prim's Algorithm – example

## Example




- We have obtained a MST of weight 17

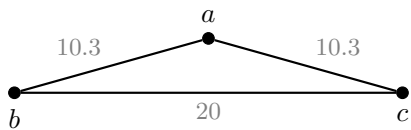
# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

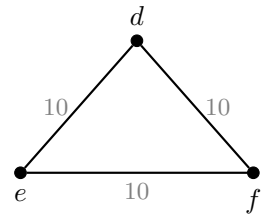
# Shortest network

- Network: a connected graph
- A shortest network is a connected network of least total weight.

# MST




$T_1$




$T_2$

- Edge lengths are drawn to scale since the angle created by the edges will play a large role in determining where to place shortcuts
- The minimum spanning tree consists of the two edges of length $10.3$ in $T_1$, and any two edges in $T_2$.
- Could we do better than a minimum spanning tree?

# Fermat point

- A similar question was posed by the 17th century French mathematician Pierre de Fermat in a letter to Evangelista Torricelli, an Italian physicist and mathematician
- In his letter, Fermat challenged Torricelli to find a point that minimizes the sum of the distances to the three vertices of a triangle.

**Definition**

A *Fermat point* of a triangle is a point $p$ for which the sum of the distances from $p$ to the three vertices is minimized. When all angles of the triangle are less than $120°$, the three angles formed at $p$ are each $120°$.

# Shortest network for a triangle

**Theorem**

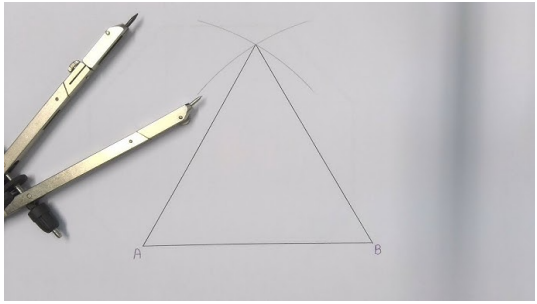
*Given three points and the triangle $T$ determined by them, the shortest network connecting the three points is either*

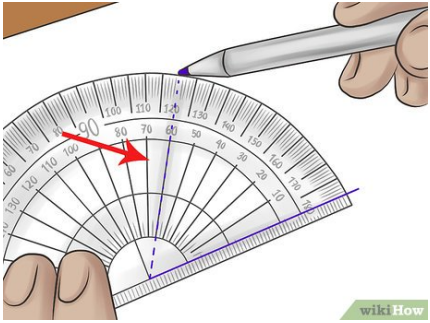
- *the two shortest sides of $T$ provided $T$ has one angle of at least $120^\circ$*
- *the three segments connecting the Fermat point for $T$ to the original three vertices of $T$*

# Torricelli's Construction

- Input: Triangle $T = \triangle abc$ where all angles measure less than $120^\circ$
- Steps
  1. Along edge $ab$ of $T$ construct an equilateral triangle using that edge and a new point $x$ that is on the opposite side of the edge as $c$
  2. Repeat step 1 for the other two edges of $T$, introducing new points $y$ and $z$ across from $b$ and $a$, respectively
  3. Join $x$ and $c$, $y$ and $b$, and $z$ and $a$ by a line segment
  4. The common intersection point of these three line segments is the Fermat point $p$ of $T$.
  5. The shortest network is the line segments joining each of the original vertices, $a, b$, and $c$, with $p$
- Output: Fermat point $p$ and shortest network connecting $a, b$, and $c$

# Torricelli's Construction – tools



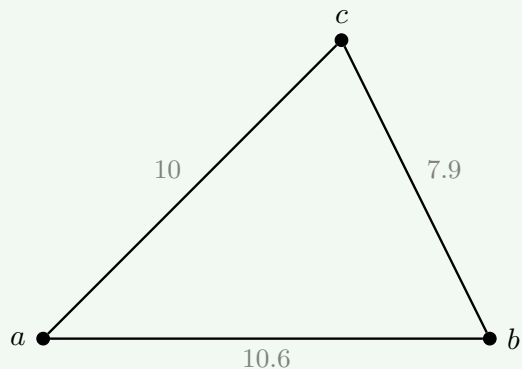




**Note**

Can be done using a ruler and a compass (or a protractor)

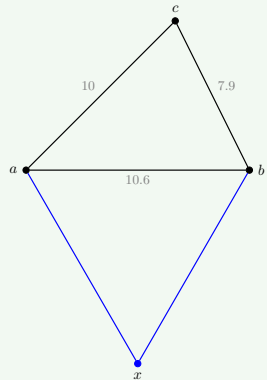
# Torricelli's Construction – example

**Example**



# Torricelli's Construction – example

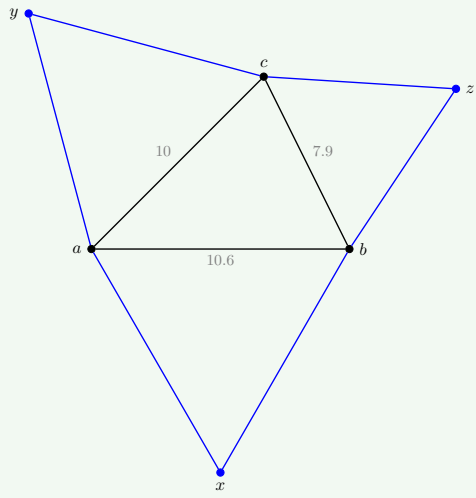
## Example




- Step 1. Form an equilateral triangle off edge $ab$ with new point $x$ on the opposite side of $ab$ as $c$

# Torricelli's Construction – example

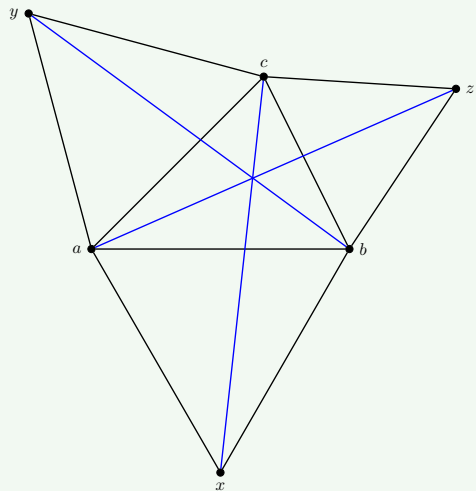
## Example




- Step 2. Repeat the construction for edges $ac$ and $bc$.

# Torricelli's Construction – example

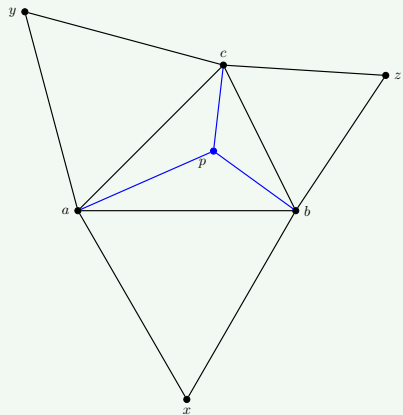
## Example




- Step 3. Join $x$ and $c$, $y$ and $b$, and $z$ and $a$

# Torricelli's Construction – example

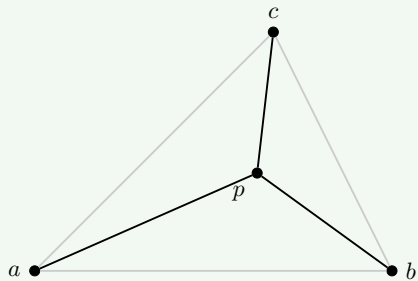
## Example




- Step 4. Find the Fermat point $p$ as the common intersection point of the three constructed segments.
- Step 5. Highlight the edges from $p$ to each of the original vertices

# Torricelli's Construction – example

## Example



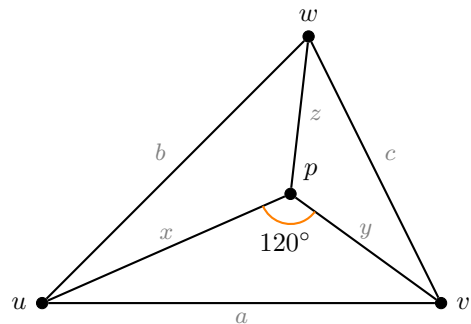

- Output: the shortest network consists of the three segments joining $p$ to the original vertices.

# Torricelli's Construction – length of the shortest network

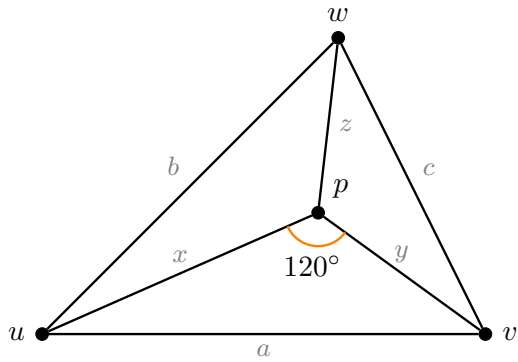
- Law of cosines

$$\begin{aligned} a^2 &= b^2 + c^2 - 2bc\cos(\angle uwv) \\ c^2 &= a^2 + b^2 - 2ab\cos(\angle wuv) \\ b^2 &= a^2 + c^2 - 2ac\cos(\angle wvu) \end{aligned}$$

- Compute the angles of the triangle
- Assign coordinates to vertices
- Solve for the lengths $x$, $y$, and $z$



# Torricelli's Construction – length of the shortest network




- If $\ell$ denotes the length of the shortest network connecting vertices of a triangle having angles that measure less than $120^\circ$ each and sides of lengths $a, b, c$

$$\ell = x + y + z = \sqrt{\frac{a^2+b^2+c^2}{2} + \frac{\sqrt{3}}{2}\sqrt{2a^2b^2+2a^2c^2+2b^2c^2-(a^4+b^4+c^4)}}$$

# Torricelli's Construction – length of the shortest network

## Example

- Continuing from the previous example

$$a = 7.9, \quad b = 10, \quad c = 10.6$$

- With the formula we get the length of the network obtained

$$\ell = \sqrt{\frac{274.77}{2} + \frac{\sqrt{3}}{2}\sqrt{48978.7752 - 26519.7777}} \approx 16.345$$

- In comparison, the MST has a total length of $17.9$
- The shortest network saves $1.555$, or $8.69\%$

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Steiner Trees

**Definition**

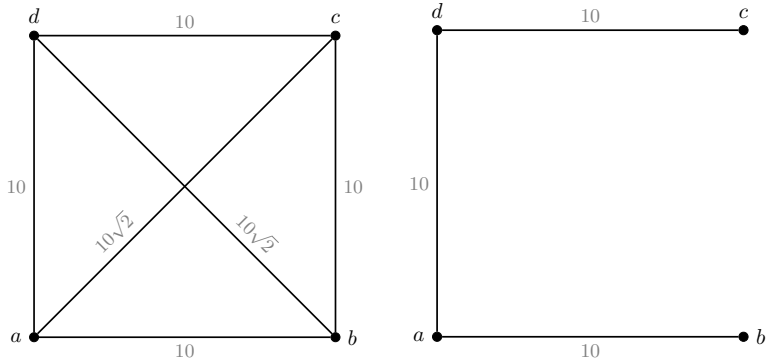
- For a graph $G$, a *Steiner point* is a new point $p$ added to the graph with degree $3$, such that the three edges incident to $p$ meet at angles of $120^\circ$.
- A *Steiner tree* is a tree whose vertices consist of the original vertices of $G$ together with possible Steiner points.

- A Steiner point is similar to a Fermat point in that it is added to a graph to help find a shortest network connecting the original vertices.
- It can be shown that finding a shortest network amounts to finding a minimum Steiner tree.
- This problem is named for the 19th century Swiss mathematician Jakob Steiner

# Finding Steiner Trees

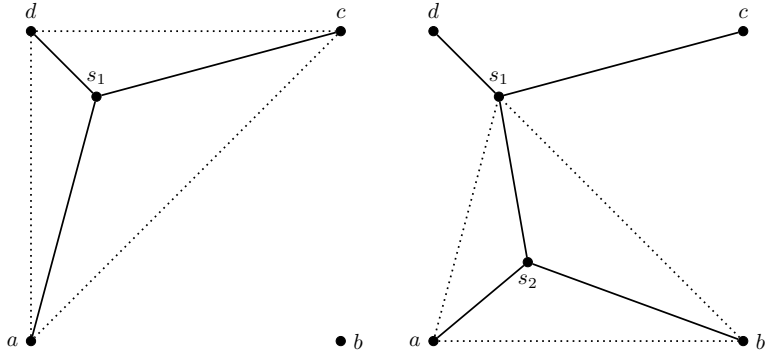
- Although the Steiner Tree Problem sounds fairly simple, it is NP-hard.
  - Finding an optimal solution can be computationally difficult.
  - No efficient algorithm is known for solving the problem in general.

# Four vertices – MST



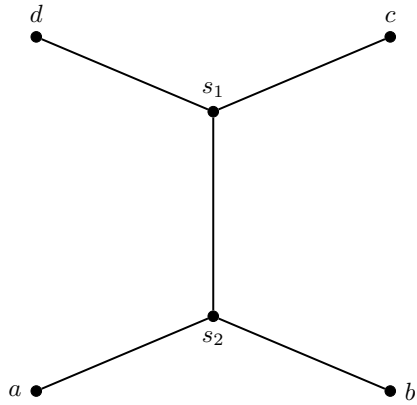

- A graph with four vertices that lie on a square
- A minimum spanning tree consists of three of the four edges of length $10$.

# Four vertices – Fermat points




- $s_1$: Fermat point for $\triangle adc$
- $s_2$: Fermat point for $\triangle abs_1$
- Length of network: $\approx 27.852$
- These are not Steiner points, since the incident edges do not form $120^\circ$ angles.

# Four vertices – Steiner tree




- Total length $27.321$
- Network with Fermat points: $27.852$
- MST: $30$

# Steiner Network Method

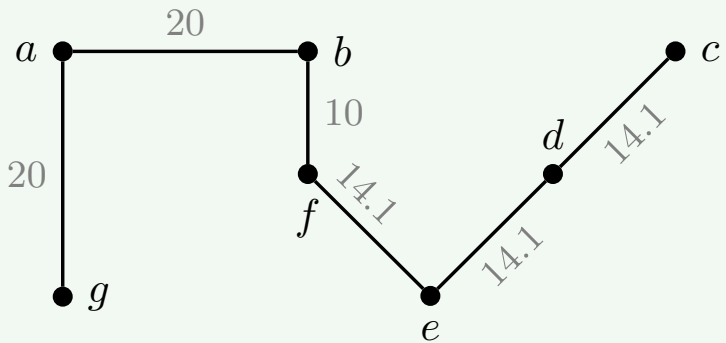
- We will not attempt to find a minimum Steiner tree.
- Instead, we use the ideas behind Steiner trees to find a network shorter than a minimum spanning tree, if possible.

# Steiner Network Method

- Step 1. Find the MST of the network
- Step 2. Form a triangle using two edges of the current network that share a vertex. If all angles of this triangle are less than $120^\circ$, find its Fermat point.
- Step 3. Update the network by removing the two edges used in Step 2 and adding edges from the corresponding vertices to the Fermat point.
- Step 4. Repeat steps 2 and 3 until all possible triangles have been considered

# Steiner Network Method – example

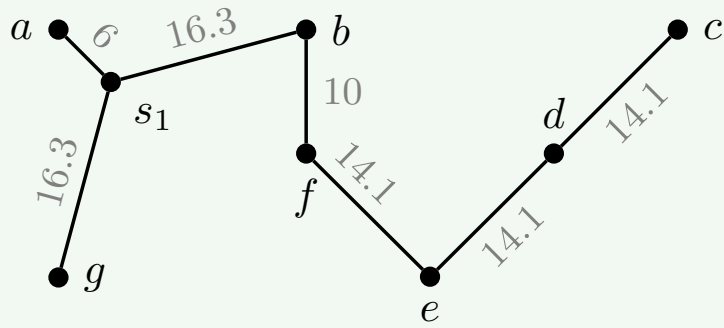
**Example**




- We begin with vertices $a, b, g$, since the angle at $a$ is $90°$.

# Steiner Network Method – example

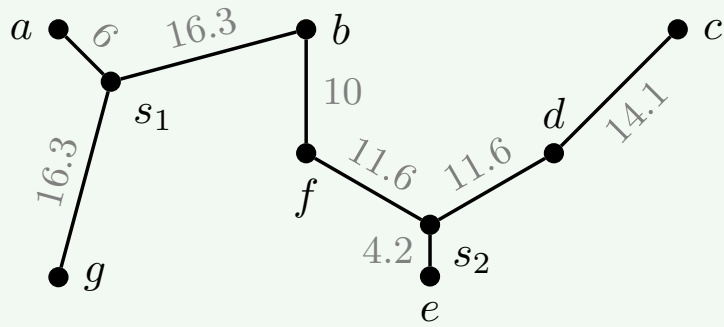
## Example




- We begin with vertices $a, b, g$. Using Torricelli's Construction, we obtain the point $s_1$.

# Steiner Network Method – example

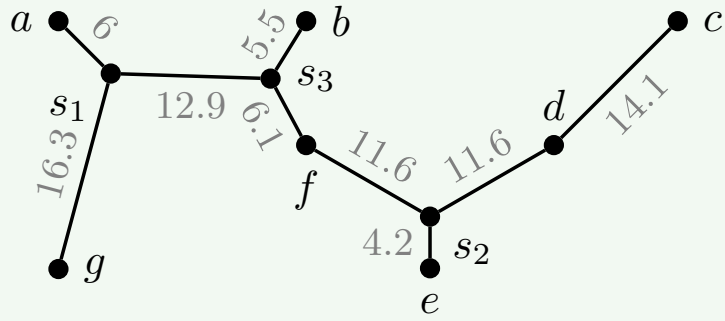
Example




- Next, consider vertices $f, e, d$. Since the angle at $e$ is $90°$, Torricelli's Construction gives the point $s_2$.
- At this stage, the angle conditions leave only one triangle to consider: $b, f, s_1$.

# Steiner Network Method – example

Example




- Finally, using vertices $b, f, s_1$, we obtain the point $s_3$.
- Total length: $88.3$
- Original (MST) length: $92.3$

# Remark

- Although we have not fully solved the optimization problem for networks containing more than three points,
- the Steiner Network Method provides a quick and simple way to identify possible improvements to a minimum spanning tree.

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

# Traveling Salesman Problem

- How should a delivery service plan its route through a city to ensure each customer is reached?
- A traveling salesman has customers in numerous cities. He must visit each of them and return home, but wishes to do this with the least total cost
- The Traveling Salesman Problem asks for a shortest cycle that visits every city exactly once and returns to the starting city; in graph terms, this is a shortest Hamiltonian cycle in a weighted complete graph.

# Metric Traveling Salesman Problem

- Abbreviated as mTSP
- Only considers the scenarios where the weights satisfy the *triangle inequality*
- For a weighted complete graph $K = (V, E, \omega)$, given any three vertices $x, y, z$

$$\omega(xy) + \omega(yz) \geq \omega(xz)$$

# mTSP Algorithm

- mTSP Algorithm combines three ideas we have studied so far – Eulerian circuits, Hamiltonian cycles and MST
- We duplicate every edge of a minimum spanning tree, making all vertex degrees even and thus producing a graph with an Eulerian circuit.
- We then convert this Eulerian circuit into a Hamiltonian cycle by skipping repeated vertices.
- This procedure is guaranteed to produce a Hamiltonian cycle only when the underlying graph is complete.
- It may still find a proper Hamiltonian cycle when the graph is not complete, but cannot be guaranteed to do so.

# mTSP Algorithm

- Input: weighted complete graph $K_n$, where the weight function $\omega$ satisfies the triangle inequality
- Steps:
  1. Find a MST $T$ for $K_n$
  2. Duplicate all the edges of $T$ to obtain $T^*$
  3. Find an Eulerian circuit for $T^*$
  4. Convert the Eulerian circuit into a Hamiltonian cycle by skipping any previously visited vertex (except for the starting and ending vertex)
  5. Calculate the total weight
- Output: Hamiltonian cycle for $K_n$

# mTSP Algorithm – example

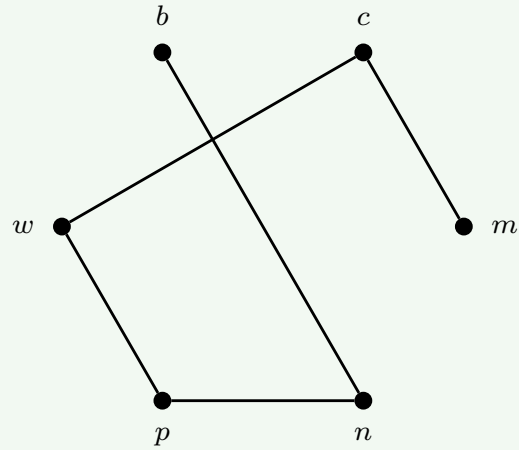
## Example

Alice must visit six cities and needs to minimize the total distance

| | Boston | Charlotte | Memphis | New York | Philadelphia | D.C. |
|---|---|---|---|---|---|---|
| Boston | . | 840 | 1316 | 216 | 310 | 440 |
| Charlotte | 840 | . | 619 | 628 | 540 | 400 |
| Memphis | 1316 | 619 | . | 1096 | 1016 | 876 |
| New York City | 216 | 628 | 1096 | . | 97 | 228 |
| Philadelphia | 310 | 540 | 1016 | 97 | . | 140 |
| Washington, D.C. | 440 | 400 | 876 | 228 | 140 | . |

# mTSP Algorithm – example

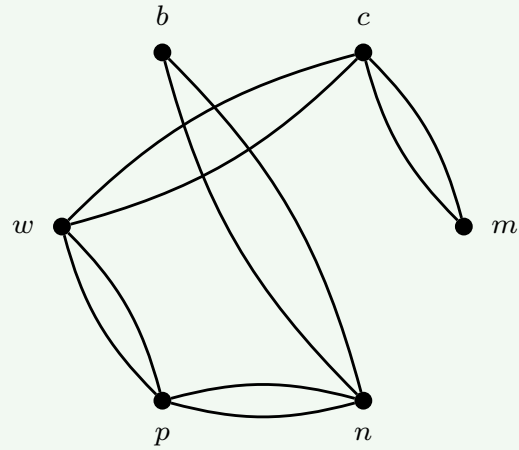
## Example




- Step 1. MST

# mTSP Algorithm – example

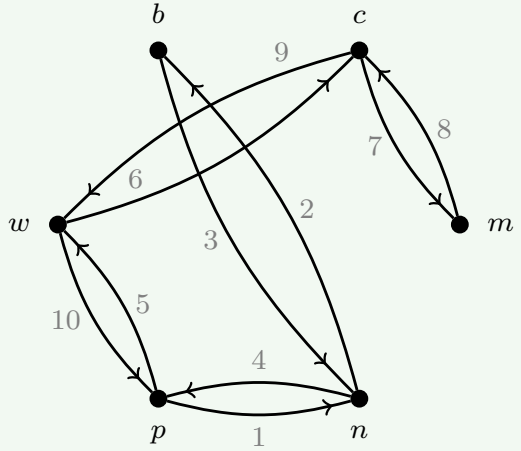
## Example




- Step 2. Duplicate all edges of the MST

# mTSP Algorithm – example

## Example



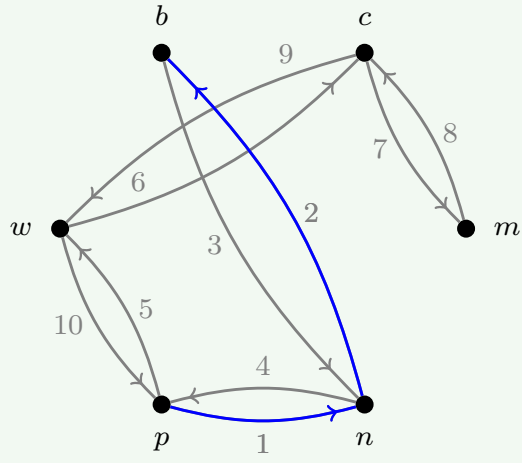

- Step 3. An Eulerian circuit is $pnbnpwcmcwp$

# mTSP Algorithm – example

## Example

Step 4

- Follow the Eulerian circuit until we reach $b$.
- Since we want a Hamiltonian cycle, we must skip any vertex that has already been visited, so we do not return to $n$.
- The next vertex along the circuit that has not been previously visited is $w$

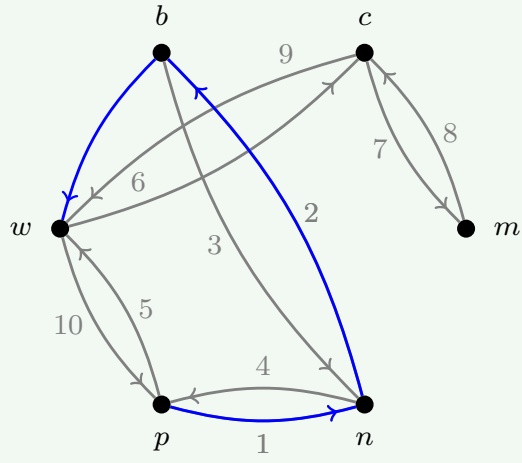


# mTSP Algorithm – example

## Example

Step 4

- Add the edge from $b$ to $w$

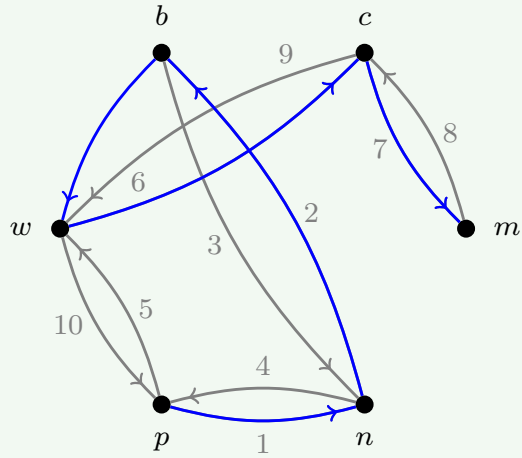


# mTSP Algorithm – example

## Example

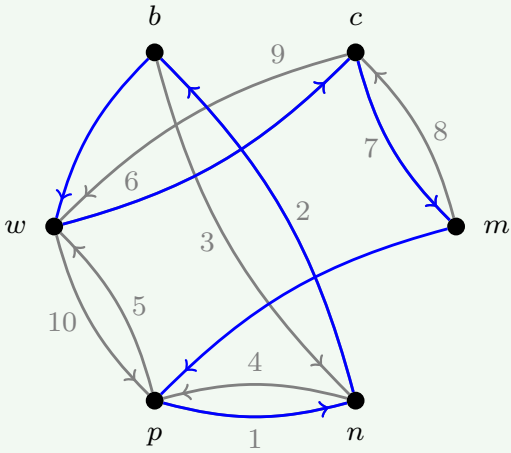
Step 4

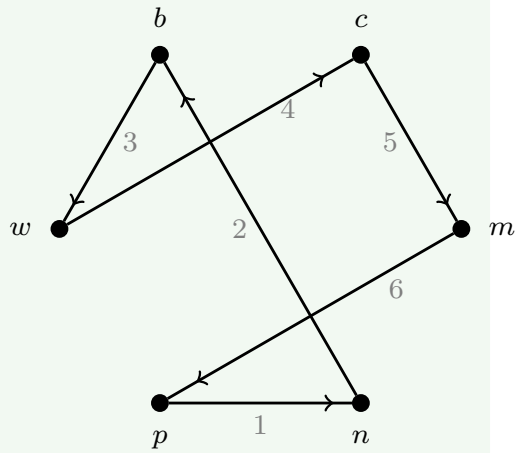
- Continue following the Eulerian circuit until $m$ is reached.
- Again, we skip $c$ because it has already been visited, and at this stage the only remaining move is to return to $p$.



# mTSP Algorithm – example

## Example



# mTSP Algorithm – example

**Example**

- Total weight of the MST: $1472$ – the worst possible Hamiltonian cycle that can arise from it will have weight at most two times that $2944$, due to the doubling of the edges
- Total weight of the Hamiltonian cycle: $2788$
- The actual optimal Hamiltonian cycle has total weight $2781$.
- Thus, the relative error of our Hamiltonian cycle is approximately $0.25\%$.

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

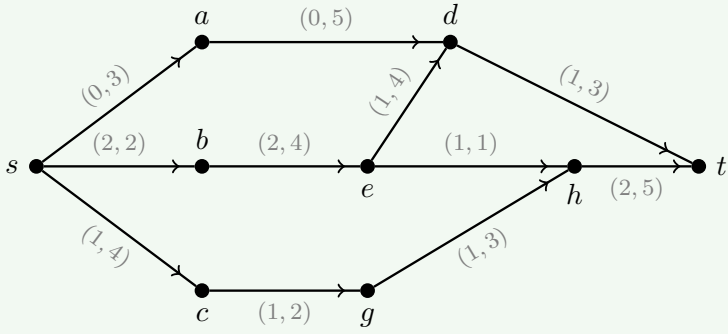
# A different notion of a network

## Definition

- A *network* is a connected digraph where each arc $e$ has an associated nonnegative real number $c(e)$, called a *capacity*.
- The network has a designated starting vertex $s$, called the *source*, and a designated ending vertex $t$, called the *sink*.
- A *flow* $f$ is a function that assigns a value $f(e)$ to each arc $e$ of the network

# Network – example

## Example




- Each arc is given a two-part label, $(f(e), c(e))$
- The first component is the flow along the arc
- The second component is the capacity

# Network

- The terms source and sink are motivated by the analogy of water flowing through a system of pipes, from the source to the sink.
- In this analogy, flow travels in the direction of the arcs, no arc can carry more than its capacity, and for every intermediate vertex the total inflow equals the total outflow.

# Feasible flow

## Definition

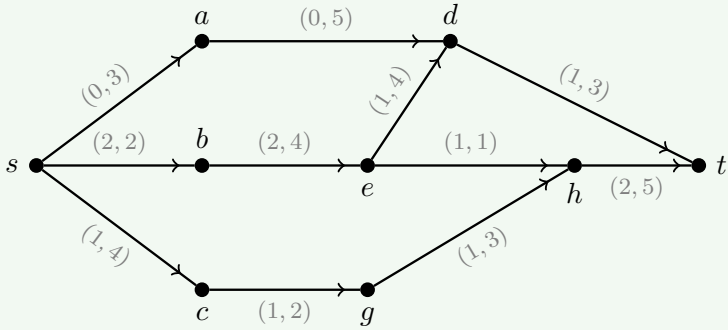
For a vertex $v$, let $f^-(v)$ represent the total flow entering $v$ and $f^+(v)$ represent the total flow exiting $v$. A flow is *feasible* if it satisfies the following conditions

- $f(e) \geq 0$ for all arcs $e$
- $c(e) \geq f(e)$ for all arcs $e$
- $f^+(v) = f^-(v)$ for all vertices other than $s$ and $t$
- $f^-(s) = f^+(t) = 0$

- The notation for inflow and outflow parallels that for in-degree and out-degree, except that here we sum the flow values on the arcs entering or leaving a vertex.
- The requirement that a flow is non-negative indicates the flow must travel in the direction of the arc, as a negative flow would indicate items going in the reverse direction
- The final condition is not necessary in theory, but more logical in practice and simplifies our analysis of flow problems

# Network – example

## Example




- $f^+(s) = 0 + 2 + 1 = 3,\ f^-(t) = 1 + 2 = 3$
- $c(ad) = 5,\ f(ad) = 0,\ c(ad) > f(ad)$
- $f^+(b) = f^-(b) = 2$

# Maximum flow

**Definition**

- The *value* of a flow is defined as

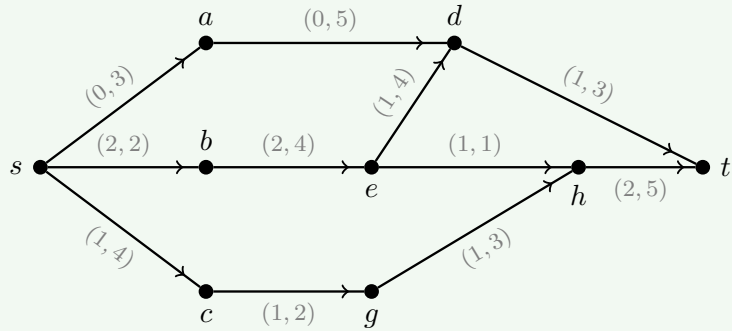
$$|f| = f^+(s) = f^-(t),$$

  i.e. the amount exiting the source which must also equal the flow entering the sink
- A *maximum flow* is a feasible flow of largest value

- In practice, we use integer values for the capacity and flow, though this is not required.
- How can we find a maximum flow?

# Network – example

## Example




- Feasible flow, value $3$
- Finding maximum flow is not as simple as putting every arc at capacity – e.g. if we had a flow of $5$ along arc $ad$, flow along $dt$ would be $6$, more than its capacity

# Slack

## Definition

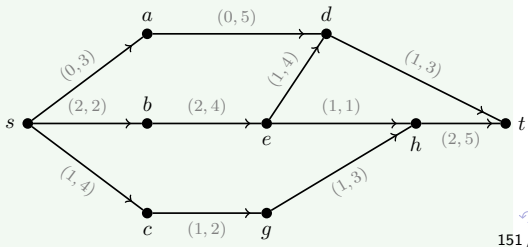
Let $f$ be a flow along a network. The *slack* $k$ of an arc is the difference between its capacity and flow

$$k(e) = c(e) - f(e).$$

*Slack edge*: $k(e) > 0$

## Example

- $k(sa) = 3$, $k(sc) = 3$, $k(sb) = 0$
- We may want to increase flow along $sa$ and $sc$ but not $sb$

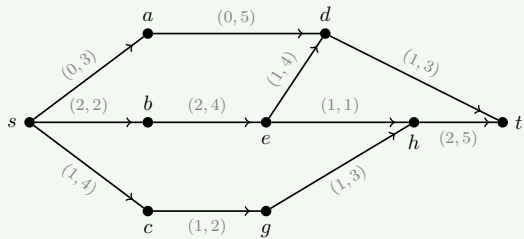


# Chain

## Definition

A *chain* $K$ is a path in a digraph where the direction of the arcs are ignored

## Example




- Both $sadt$ and $sadeht$ are chains
- $sadeht$ is not a directed path

# Augmenting Flow Algorithm (Ford-Fulkerson Algorithm)

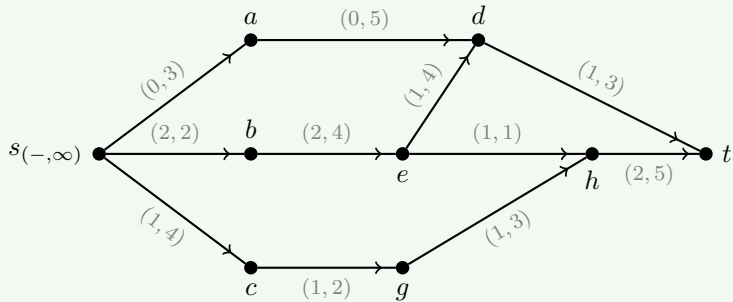
- This algorithm is similar in spirit to Dijkstra's Algorithm, which finds shortest paths in a graph or digraph.
- Vertices are assigned two-part labels that help us construct a chain along which the flow can be increased.
- The label consists of two parts, the second component is denoted by $\sigma(v)$
- Input: Network $G = (V, E, c)$, where each arc is given a capacity $c$, a designated source $s$ and sink $t$, and an initial feasible flow $f$
- Output: Maximum flow $f$

# Augmenting Flow Algorithm – steps

1. Label $s$ with $(-, \infty)$, set $\sigma(v) = \infty$ for other vertices
2. Choose a labeled vertex $x$, scan $x$:
   a. For any arc $yx$, if $f(yx) > 0$ and $y$ is unlabeled, then label $y$ with $(x^-, \sigma(y))$, where $\sigma(y) = \min\{\sigma(x), f(yx)\}$
   b. For any arc $xy$, if $k(xy) > 0$ and $y$ is unlabeled, then label $y$ with $(x^+, \sigma(y))$, where $\sigma(y) = \min\{\sigma(x), k(xy)\}$
3. If $t$ has been labeled, go to Step 4. Otherwise, choose another labeled vertex that has not yet been scanned and return to Step 2. If all labeled vertices have been scanned and $t$ is still unlabeled, then $f$ is a maximum flow.
4. Find an $s$-$t$ chain $K$ by backtracking from $t$ to $s$ using the vertex labels. Along the edges of $K$, increase the flow by $\sigma(t)$ units if they are in the forward direction and decrease by $\sigma(t)$ units in they are in the backward direction. Remove all vertex labels except that of $s$ and return to Step 2

# Augmenting Flow Algorithm – example

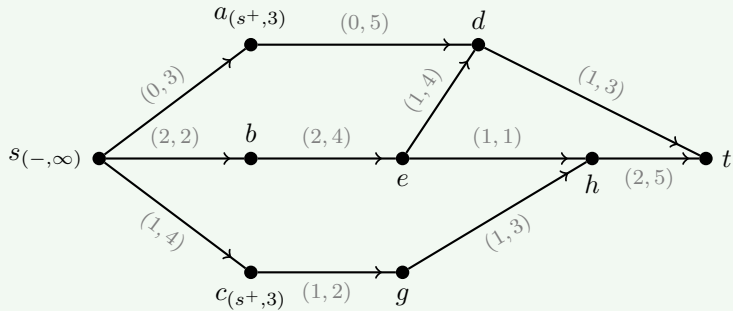
## Example




- Step 1. Label $s$ as $(-, \infty)$

# Augmenting Flow Algorithm – example

## Example

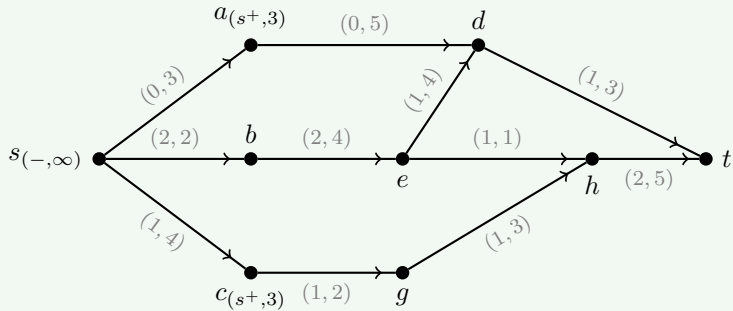



Step 2.

- There are no arcs to $s$.
- Arcs out of $s$: $sa, sb, sc$, with slack $3, 0, 3$
- Label $a$ with $(s^+, 3)$, $c$ with $(s^+, 3)$. Leave $b$ unlabeled since $sb$ has slack $0$

# Augmenting Flow Algorithm – example

## Example

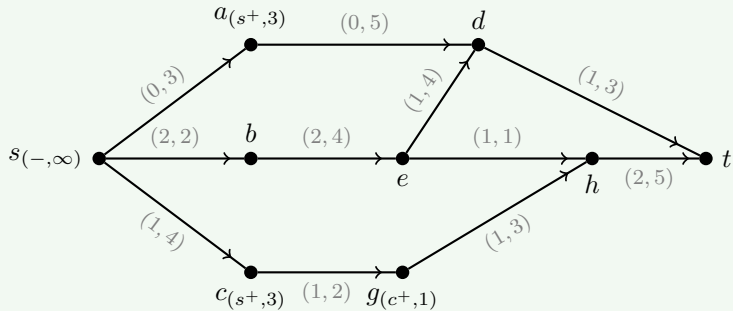



Step 3.

- Since $t$ is not labeled, choose another labeled vertex that has not yet been scanned —either $a$ or $c$.
- Choose $c$.

# Augmenting Flow Algorithm – example

## Example

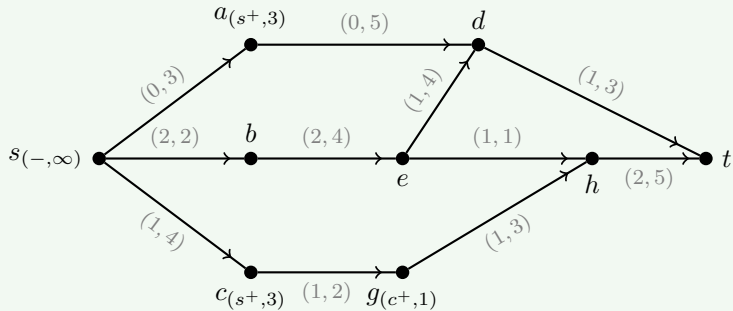



Step 2.

- The only arc entering $c$ comes from a labeled vertex.
- Consider the arcs out of $c$ – there is only $cg$ with slack of $1$
- Label $g$ as $(c^+, 1)$, where $\sigma(g) = \min\{\sigma(c), k(cg)\} = \min\{3, 1\} = 1$

# Augmenting Flow Algorithm – example

## Example

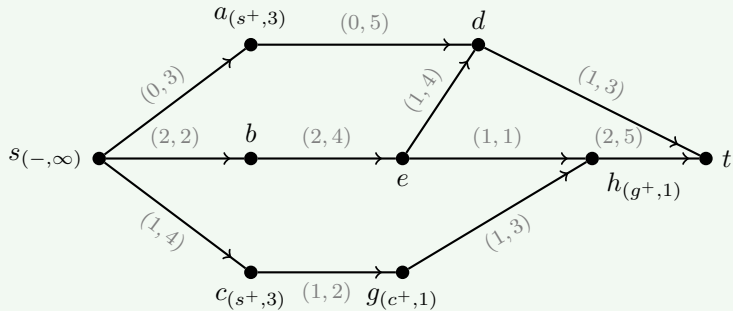



Step 3.

- $t$ is not labeled
- We can scan either $a$ or $g$
- Choose $g$

# Augmenting Flow Algorithm – example

## Example

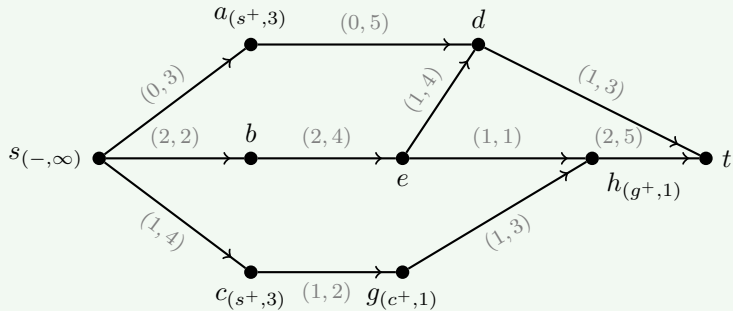



Step 2.

- The only arc entering $g$ comes from a labeled vertex.
- The only arc out of $g$ is $gh$
- Label $h$ as $(g^+, 1)$, where $\sigma(h) = \min\{\sigma(g), k(gh)\} = \min\{1, 2\} = 1$

# Augmenting Flow Algorithm – example

## Example

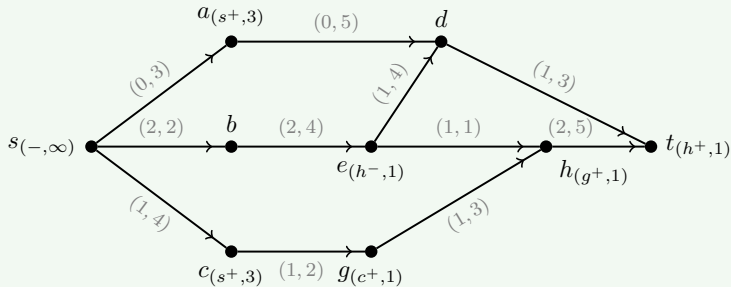



Step 3.

- $t$ is not labeled, we can choose $a$ or $h$
- Choose $h$

# Augmenting Flow Algorithm – example

## Example

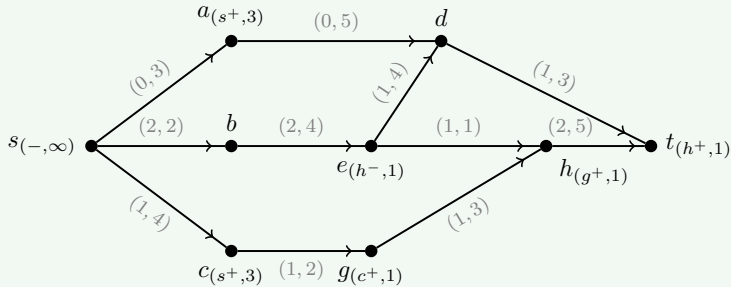



Step 2.

- There is one unlabeled vertex with an arc into $h$, namely $e$. Label $e$ with $(h^-, 1)$, since $\sigma(e) = \min\{\sigma(h), f(eh)\} = \min\{1, 1\} = 1$.
- Label $t$ as $(h^+, 1)$, where $\sigma(t) = \min\{\sigma(h), k(ht)\} = \min\{1, 3\} = 1$

# Augmenting Flow Algorithm – example

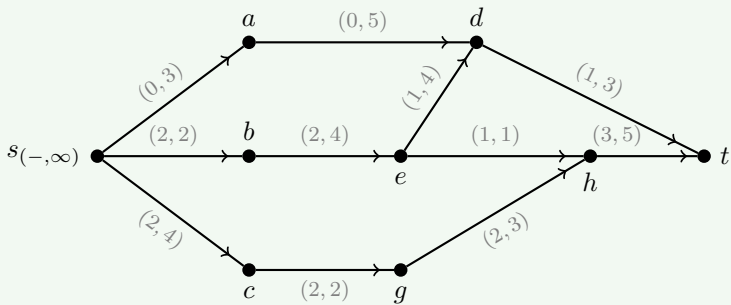
## Example




- Step 3. $t$ is labeled, go to Step 4
- Step 4. Backtracking from $t$ to $s$ using slack edges gives the chain $scght$.

# Augmenting Flow Algorithm – example

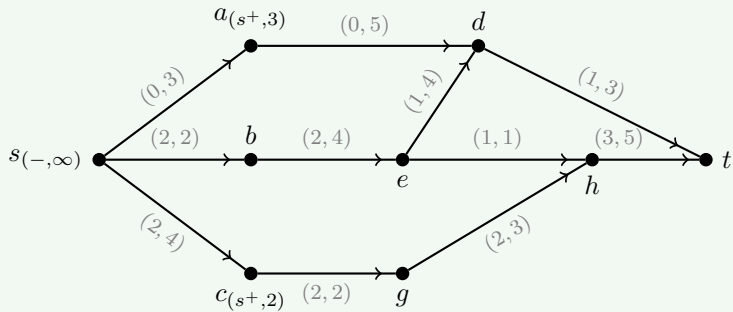
## Example




- Step 4. Backtracking from $t$ to $s$ using slack edges gives the chain $scght$
  - Increase the flow by $\sigma(t) = 1$ units along each of these edges since all are in the forward direction.
  - Update the network flow and remove all labels except for $s$

# Augmenting Flow Algorithm – example

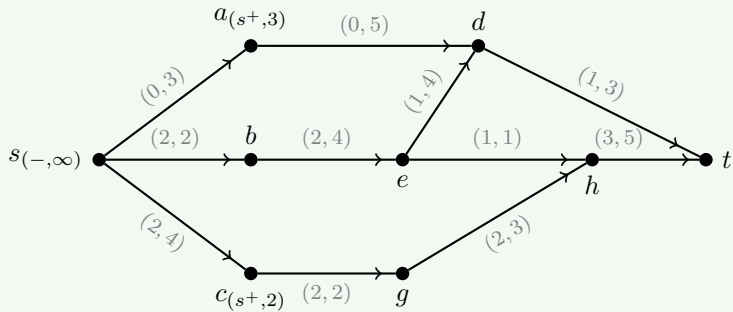
## Example




- Step 2. Label $a$ with $(s^+, 3)$ and $c$ with $(s^+, 2)$
- Step 3. Choose $c$ to scan

# Augmenting Flow Algorithm – example

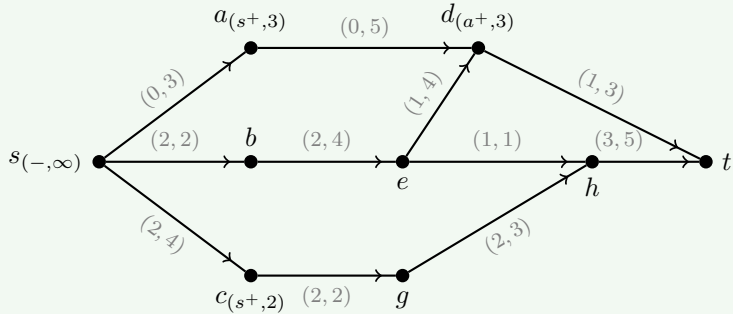
## Example




- Step 2. No vertex to label, $k(cg) = 0$
- Step 3. Choose $a$ to scan

# Augmenting Flow Algorithm – example

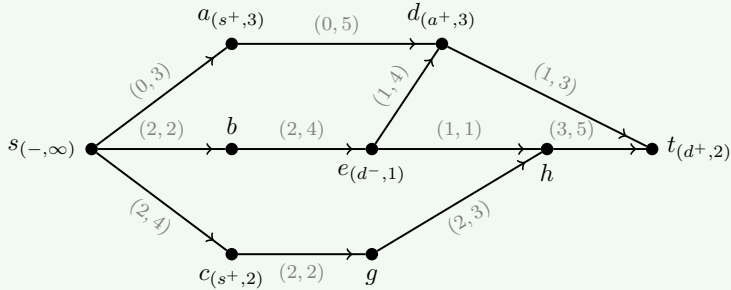
## Example




- Step 2. Label $d$ with $(a^+, 3)$, where $\sigma(d) = \min\{\sigma(a), k(ad)\} = \min\{3, 5\} = 3$
- Step 3. Only unscanned labeled vertex is $d$

# Augmenting Flow Algorithm – example

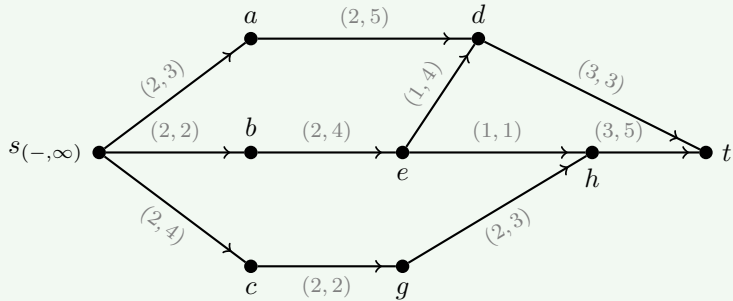
## Example




- Step 2. Label $e$ with $(d^-, 1)$, where $\sigma(e) = \min\{\sigma(d), f(ed)\} = \min\{3, 1\} = 1$. Label $t$ with $(d^+, 2)$, $\sigma(t) = \min\{\sigma(d), k(dt)\} = \min\{3, 2\} = 2$
- Step 3. $t$ is labeled, go to step 4
- Step 4. Backtracking gives the chain $sadt$

# Augmenting Flow Algorithm – example

## Example

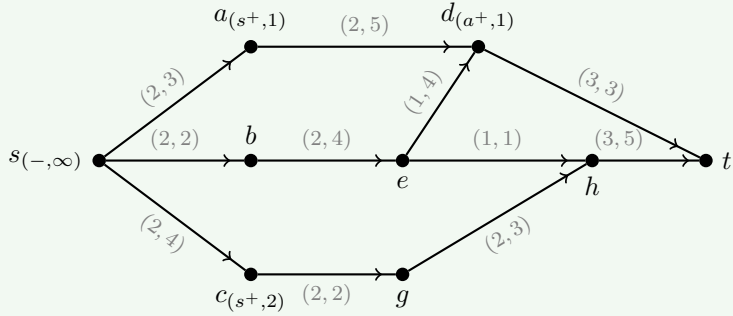



Step 4.

- Find chain $sadt$, increase the flow by $\sigma(t) = 2$ units along each of these edges since all are in the forward direction
- Update the network flow and remove all labels except for $s$

# Augmenting Flow Algorithm – example

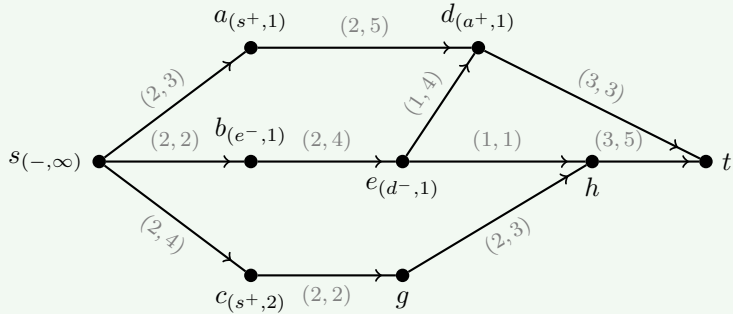
## Example




- Step 2. Label $a$ and $c$
- Step 3. Choose $a$
- Step 2. Label $d$
- Step 3. Choose $d$

# Augmenting Flow Algorithm – example

## Example




- Step 2. Label $e$ as $(d^-, 1)$. Vertex $t$ is not labeled because $k(dt) = 0$
- Step 3. Choose $e$
- Step 2. Label $b$
- Step 3. No vertices to be labeled, we get a maximum flow

# Augmenting Flow Algorithm

- When the Augmenting Flow Algorithm halts, a maximum flow has been achieved, though understanding why this flow is indeed maximum requires additional terminology and results

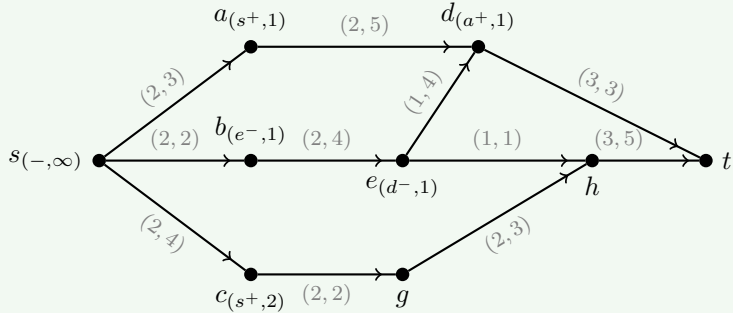
# Cut

## Definition

- Let $P$ be a set of vertices and $\overline{P} = V - P$.
- A *cut* $(P, \overline{P})$ is the set of all arcs $xy$ where $x \in P$ and $y \in \overline{P}$
- An $s - t$ *cut* is a cut in which $s \in P$, $t \in \overline{P}$

# Cut – example

## Example



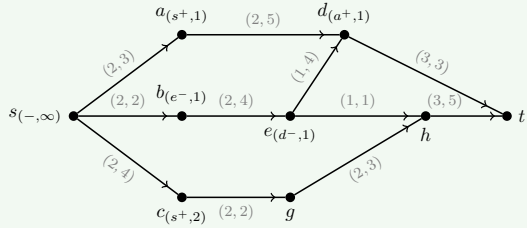

- $P = \{s, a, e, g\}$, $\overline{P} = \{b, c, d, h, t\}$
- $(P, \overline{P}) = \{sb, sc, ad, ed, eh, gh\}$
- Note: $be \notin (P, \overline{P})$ because $b \in \overline{P}$, $e \in P$

# Capacity

## Definition

The *capacity* of a cut, $c(P, \overline{P})$, is defined as the sum of the capacities of the arcs that comprise the cut.

## Example




- $P = \{s, a, e, g\}$, $\overline{P} = \{b, c, d, h, t\}$, $(P, \overline{P}) = \{sb, sc, ad, ed, eh, gh\}$, $c(P, \overline{P}) = 2 + 4 + 5 + 4 + 1 + 3 = 19$
- $P = \{s\}$, $c(P, \overline{P}) = 9$

# Max Flow-Min Cut

**Theorem**

*In any directed network, the value of a maximum $s-t$ flow equals the capacity of a minimum $s-t$ cut.*

- The difficulty in using this theorem directly is that one must still identify a minimum cut.
- We can use the vertex labeling procedure to obtain our minimum cut.

# Min-Cut Method

Steps

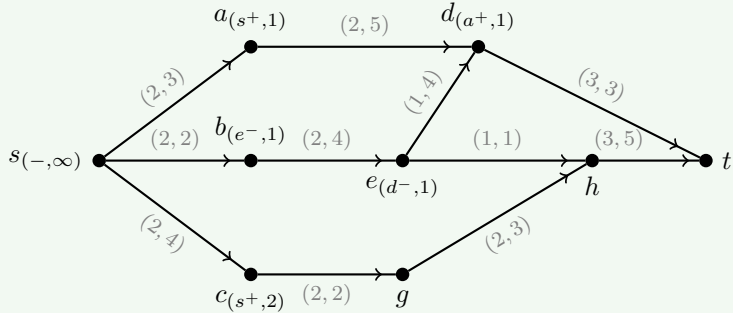
1. Let $G = (V, A, c)$ be a network with a designated source $s$ and sink $t$ and each arc is given a capacity $c$
2. Apply the Augmenting Flow Algorithm
3. Define an $s - t$ cut $(P, \overline{P})$ where $P$ is the set of labeled vertices from the final implementation of the algorithm
4. $(P, \overline{P})$ is a minimum $s - t$ cut for $G$

**Note**

In practice, we can perform the Augmenting Flow Algorithm and the Min-Cut Method simultaneously, thus finding a maximum flow and providing a proof that it is maximum (through the use of a minimum cut) in one complete procedure.

# Min-Cut Method – example

## Example




- $P = \{s, a, b, c, d, e\}$, $\overline{P} = \{g, h, t\}$
- $(P, \overline{P}) = \{dt, eh, cg\}$
- $c(P, \overline{P}) = 3 + 1 + 2 = 6$

# Shortest Paths, Project Scheduling, Trees, and Networks

- Dijkstra's Algorithm
- Chinese Postman Problem
- Project scheduling
- Critical path
- Trees
- Spanning trees
- Shortest networks
- Steiner Trees
- Metric Traveling Salesman Problem
- Flow and capacity
- Rooted trees

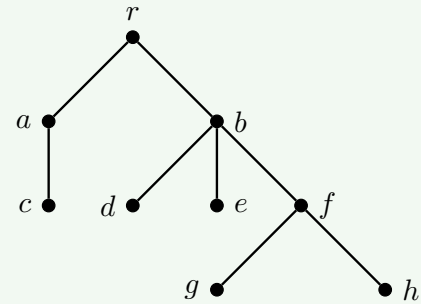
# Definition

## Definition

- A *rooted tree* is a tree $T$ with a special designated vertex $r$, called the *root*
- The *level* of any vertex in $T$ is defined as the length of its shortest path to $r$
- The *height* of a rooted tree is the largest level for any vertex in $T$

## Example

- root $r$: level $0$
- $a$, $b$: level $1$
- $c, d, e, f$: level $2$
- $g, h$: level $3$
- height of the tree: $3$



# Terminologies

## Definition

Let $T$ be a tree with root $r$. Then for any vertices $x$ and $y$

- $y \neq x$ is on the unique path from $x$ to $r$: $x$ is a *descendant* of $y$; $y$ is an *ancestor* of $x$
- $x$ is a descendant of $y$ and exactly one level below $y$: $x$ is a *child* of $y$, $y$ is a *parent* of $x$
- $x$ is a *sibling* of $y$ if $x \neq y$ and $x$ and $y$ have the same parent.

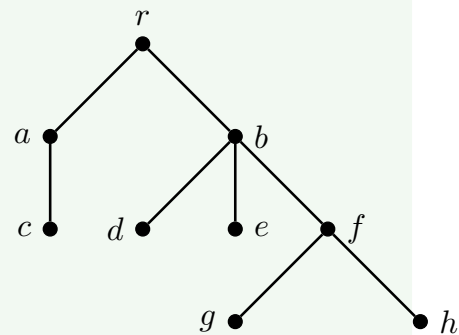
## Note

Analogy: family tree

# Terminologies – example

## Example

- parent of $a$: $r$
- child of $a$: $c$
- parent of $e$: $b$
- $e$ has no children
- ancestors of $g$: $f, b, r$ – unique path from $g$ to $r$ is $gfbr$
- descendants of $b$: $d, e, f, g, h$
- siblings of $d$: $e, f$



# Depth-First Search Tree

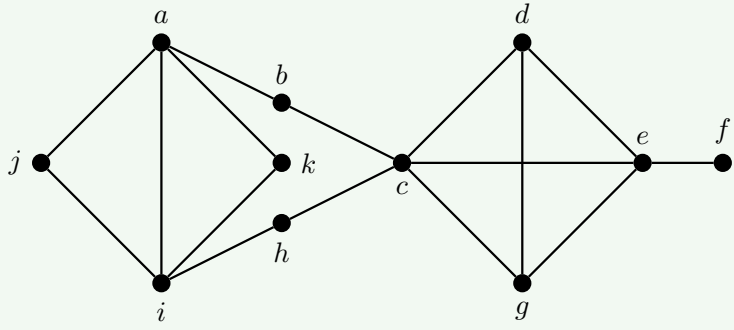
- The main idea is to follow a path as far as possible from the root before backtracking.
- If this path does not include all vertices, we backtrack and build further branches to obtain a tree.
- The formal description of the algorithm uses an ordered list of neighbors for each vertex, and this order determines which new vertex is added next.
- For simplicity, we will always use an alphabetical order when considering neighbor lists
- Input: Simple (no multi-edges or loops) connected graph $G = (V, E)$ and a designated root vertex $r$
- Output: Depth-first search tree $T$

# Depth-First Search Tree – steps

1. Initialize the DFS tree $T = (V', E')$ with $V' = \{r\}$ and $E' = \emptyset$. Set $r$ as the current vertex $v$.
2. Select the first unvisited neighbor $x$ of the current vertex $v$. Add vertex $x$ and edge $vx$ to $T$, and recursively repeat Step 2 with $x$ as the new current vertex until no unvisited neighbors remain.
3. If all vertices of $G$ are now in $T$, then $T$ is the depth-first search tree. Otherwise, backtrack along the path from the last visited vertex to the root until you find a vertex $v$ with an unvisited neighbor. Use $v$ as the current vertex and repeat Step 2.

# Depth-First Search Tree – example

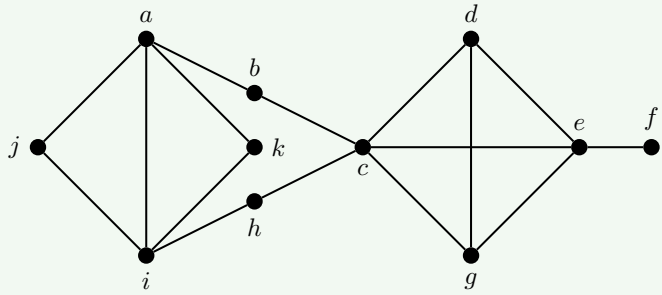
## Example




- Suppose $a$ is the root
- Step 1. Current vertex is $a$
- Step 2. Add $b, c, d, e, f$ in order. This process stops at $f$, since $f$ has no unvisited neighbors in $G$.

# Depth-First Search Tree – example

## Example



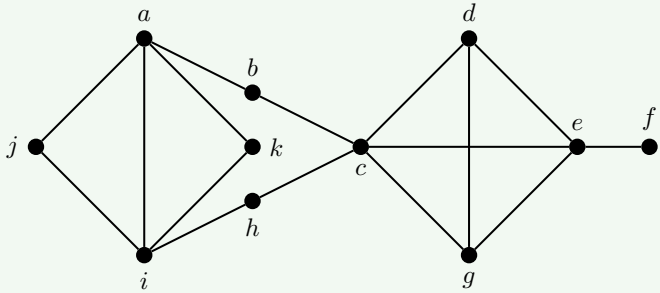


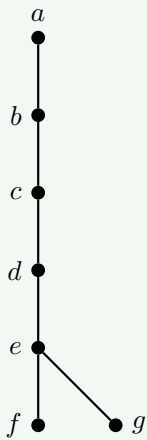


- Step 3. Backtracking along the path, the first vertex with an unvisited neighbor is $e$
- Step 2. Add edge $eg$ and vertex $g$ to $T$

# Depth-First Search Tree – example

## Example

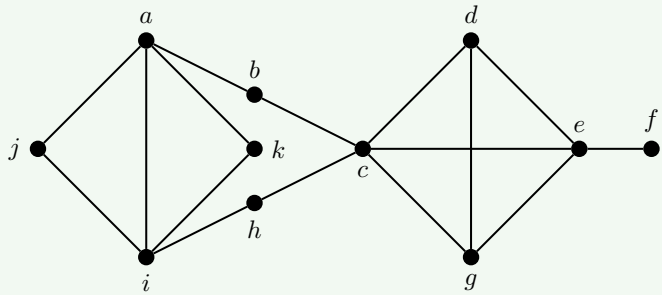


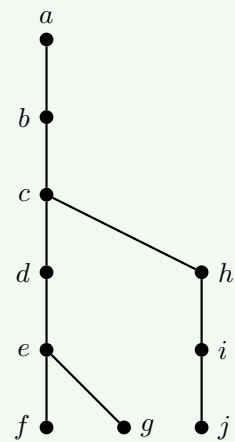




- Step 3. Backtracking along the path from $g$ to $a$, the first vertex with an unvisited neighbor is $c$
- Step 2. Add $h, i, j$ in order.

# Depth-First Search Tree – example

## Example

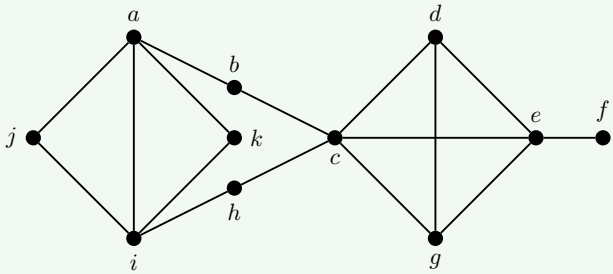


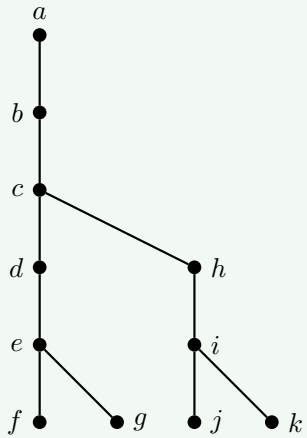




- Step 3. Backtracking along the path from $j$ to $a$, the first vertex with an unvisited neighbor is $i$
- Step 2. Add $k$

# Depth-First Search Tree – example

## Example




- $T$ contains all the vertices of $G$
- The resulting tree is the depth-first search tree
- The tree has height $5$, with one vertex at each of levels $1$ and $2$, two vertices at each of levels $3$ and $4$, and four vertices at level $5$.

# Remark

- If the graph is not connected, we can slightly adjust the algorithm and get a forest as the output
- Search for unvisited vertices when no vertex with unvisited neighbors can be found

# Breadth-First Search Tree

- Main objective is to add as many neighbors of the root as possible in the first step
- At each subsequent step, we add all unvisited neighbors of the most recently added vertices.
- As with depth-first search, we will use alphabetical ordering of neighbor lists.
- Input: Simple connected graph $G = (V, E)$ and a designated root vertex $r$
- Output: Breadth-first tree $T$

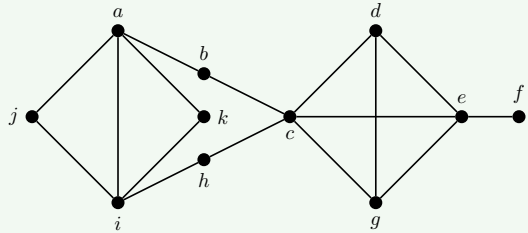
# Breadth-First Search Tree – steps

1. Initialize the BFS tree $T = (V', E')$ with the root vertex $r$, i.e., $V' = \{r\}$, and mark $r$ as visited.
2. Add all neighbors of $r$ to $V'$, and add the corresponding edges from $r$ to each neighbor to $E'$. Mark all these neighbors as visited. Let this set of newly added vertices be the current level.
3. For each vertex $v$ in the current level (in alphabetical order):
   - Add all unvisited neighbors $x$ of $v$ to $V'$.
   - Add the edge $(v, x)$ to $E'$.
   - Mark each such neighbor $x$ as visited.

   Let the collection of all such newly added vertices form the next level.
4. If $T$ now includes all vertices of $G$, the process is complete. Otherwise, repeat step 3 using the next level as the current level.

# Breadth-First Search Tree – example

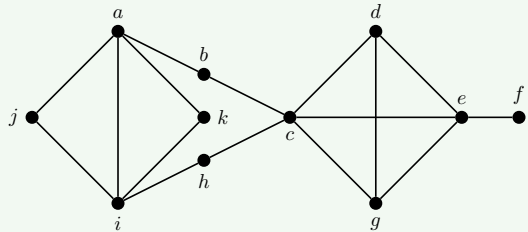
## Example




- Let's consider the same example
- Take $a$ as the root
- Step 1. Add $a, b, i, j, k$
- Step 2. Current level: $b, i, j, k$

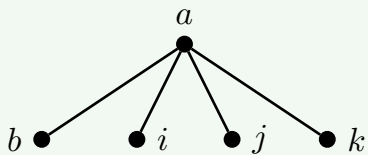
# Breadth-First Search Tree – example

## Example



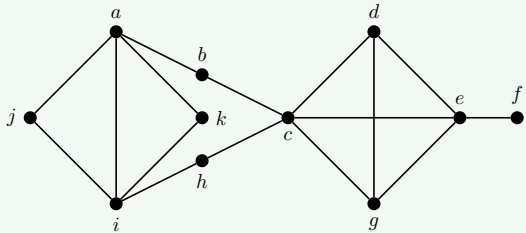

Step 3

- Vertex $b$: add neighbor $c$, and edge $bc$
- Vertex $i$: add neighbor $h$ and edge $ih$
- $j$ and $k$ do not have any unvisited neighbors

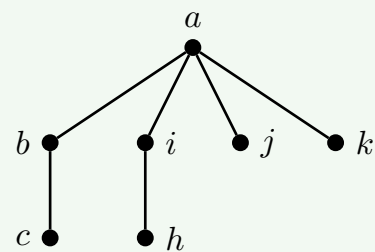


# Breadth-First Search Tree – example

## Example

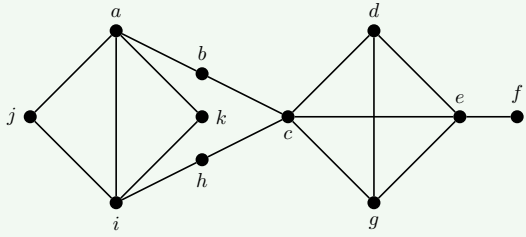



- Step 4. $T$ does not contain all vertices, repeat step 3. Current level: $c, h$
- Step 3.
  - Vertex $c$: unvisited neighbors $d, e, g$
  - Vertex $h$ has no unvisited neighbors

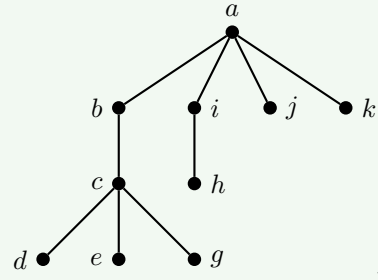


# Breadth-First Search Tree – example

## Example

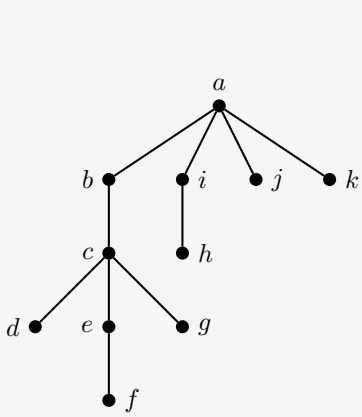


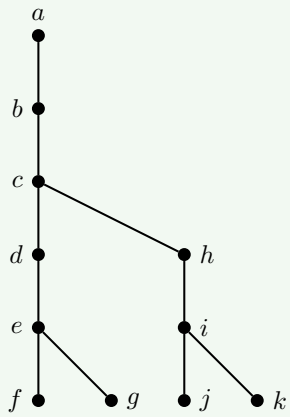

- Step 4. $T$ does not contain all vertices, repeat step 3. Current level: $d, e, g$
- Step 3.
  - Vertex $d$ has no unvisited neighbors
  - Vertex $e$: add neighbor $f$ and edge $ef$
  - Now we have all vertices



# Breadth-First Search Tree – example

## Example

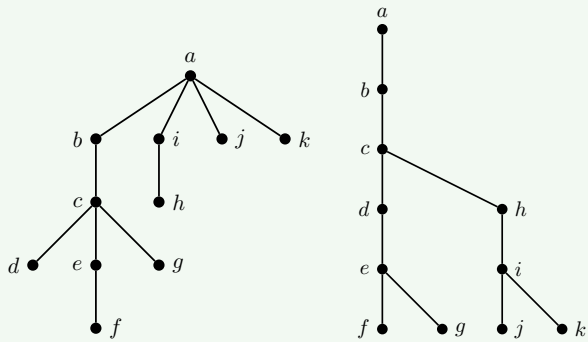



- Left: BFS tree; Right: DFS tree
- BFS trees are often shorter in height than their DFS counterparts.

# Breadth-First Search Tree – example

## Example




- The BFS tree has height $4$, with four vertices on level $1$, two on level $2$, three on level $3$, and one on level $4$.
- The DFS tree has height $5$, with one vertex at each of levels $1$ and $2$, two vertices at each of levels $3$ and $4$, and four vertices at level $5$.